

# Bits, Bytes, and Integers

15-213: Introduction to Computer Systems  
2<sup>nd</sup> and 3<sup>rd</sup> Lectures, Sep. 3 and Sep. 8, 2015

## Instructors:

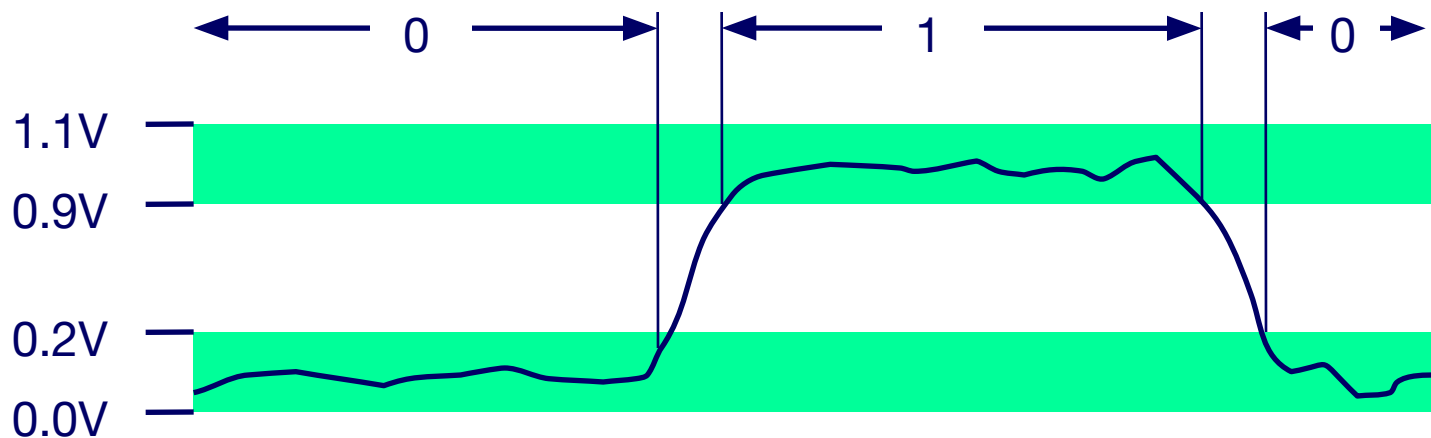
Randal E. Bryant and David R. O'Hallaron

# Today: Bits, Bytes, and Integers

- **Representing information as bits**
- **Bit-level manipulations**
- **Integers**
  - Representation: unsigned and signed
  - Conversion, casting
  - Expanding, truncating
  - Addition, negation, multiplication, shifting
  - Summary
- **Representations in memory, pointers, strings**

# Everything is bits

- Each bit is 0 or 1
- By encoding/interpreting sets of bits in various ways
  - Computers determine what to do (instructions)
  - ... and represent and manipulate numbers, sets, strings, etc...
- **Why bits? Electronic Implementation**
  - Easy to store with bistable elements
  - Reliably transmitted on noisy and inaccurate wires



# For example, can count in binary

## ■ Base 2 Number Representation

- Represent  $15213_{10}$  as  $11101101101101_2$
- Represent  $1.20_{10}$  as  $1.0011001100110011[0011]\dots_2$
- Represent  $1.5213 \times 10^4$  as  $1.1101101101101_2 \times 2^{13}$

# Encoding Byte Values

## ■ Byte = 8 bits

- Binary  $00000000_2$  to  $11111111_2$
- Decimal:  $0_{10}$  to  $255_{10}$
- Hexadecimal  $00_{16}$  to  $FF_{16}$ 
  - Base 16 number representation
  - Use characters '0' to '9' and 'A' to 'F'
  - Write  $FA1D37B_{16}$  in C as
    - `0xFA1D37B`
    - `0xfa1d37b`

| Hex | Decimal | Binary |
|-----|---------|--------|
| 0   | 0       | 0000   |
| 1   | 1       | 0001   |
| 2   | 2       | 0010   |
| 3   | 3       | 0011   |
| 4   | 4       | 0100   |
| 5   | 5       | 0101   |
| 6   | 6       | 0110   |
| 7   | 7       | 0111   |
| 8   | 8       | 1000   |
| 9   | 9       | 1001   |
| A   | 10      | 1010   |
| B   | 11      | 1011   |
| C   | 12      | 1100   |
| D   | 13      | 1101   |
| E   | 14      | 1110   |
| F   | 15      | 1111   |

# Example Data Representations

**32-bit** or **64-bit** describes how many bits the CPU uses to represent memory addresses and registers — not how big all data types are.

| sizes in bytes                                               | C Data Type        | Typical 32-bit | Typical 64-bit | x86-64            |
|--------------------------------------------------------------|--------------------|----------------|----------------|-------------------|
| also integer type                                            | <b>char</b>        | 1 <b>Byte</b>  | 1              | 1                 |
| small integer                                                | <b>short</b>       | 2              | 2              | 2                 |
| normal integer                                               | <b>int</b>         | 4              | 4              | 4                 |
| large integer                                                | <b>long</b>        | 4              | 8              | 8                 |
| 7 decimal digits                                             | <b>float</b>       | 4              | 4              | 4                 |
| 15–16 decimal digits                                         | <b>double</b>      | 8              | 8              | 8                 |
| huge                                                         | <b>long double</b> | –              | –              | 10/16 <b>Byte</b> |
| A <b>pointer</b> is a variable that stores a memory address. | <b>pointer</b>     | 4              | 8              | 8                 |

**float f = 3.14f;**

**double d = 3.141592653589793;**

## 32-bit means:

Max address =  $2^{32} \approx 4,294,967,296$  bytes  $\approx 4$  GB

Can directly address up to **~4 GB of memory**

## 64-bit means:

Max address =  $2^{64} \approx 1.8446744e+19$  **Huge**  
18,446,744,073,709,551,616

Can directly address up to **~16 exabytes of memory**

**64-bit means the machine can address more memory —  
not that all data types are bigger.**

Check the machine architecture: `uname -m`  
more information: `lscpu`

# Today: Bits, Bytes, and Integers

- Representing information as bits
- **Bit-level manipulations**
- **Integers**
  - Representation: unsigned and signed
  - Conversion, casting
  - Expanding, truncating
  - Addition, negation, multiplication, shifting
  - Summary
- Representations in memory, pointers, strings

# Boolean Algebra

## ■ Developed by George Boole in 19th Century

- Algebraic representation of logic
  - Encode “True” as 1 and “False” as 0

### And

- $A \& B = 1$  when both  $A=1$  and  $B=1$

| $\&$ | 0 | 1 |
|------|---|---|
| 0    | 0 | 0 |
| 1    | 0 | 1 |

### Or

- $A | B = 1$  when either  $A=1$  or  $B=1$

| $ $ | 0 | 1 |
|-----|---|---|
| 0   | 0 | 1 |
| 1   | 1 | 1 |

### Not

- $\sim A = 1$  when  $A=0$

| $\sim$ |   |
|--------|---|
| 0      | 1 |
| 1      | 0 |

### Exclusive-Or (Xor)

- $A \wedge B = 1$  when either  $A=1$  or  $B=1$ , but not both

| $\wedge$ | 0 | 1 |
|----------|---|---|
| 0        | 0 | 1 |
| 1        | 1 | 0 |

# General Boolean Algebras

## ■ Operate on Bit Vectors

- Operations applied bitwise

|                       |                   |                   |                   |
|-----------------------|-------------------|-------------------|-------------------|
| 01101001              | 01101001          | 01101001          |                   |
| <u>&amp; 01010101</u> | <u>  01010101</u> | <u>^ 01010101</u> | <u>~ 01010101</u> |
| 01000001              | 01111101          | 00111100          | 10101010          |

## ■ All of the Properties of Boolean Algebra Apply

# Example: Representing & Manipulating Sets

## ■ Representation

- Width  $w$  bit vector represents subsets of  $\{0, \dots, w-1\}$
- $a_j = 1$  if  $j \in A$

demo:  
ch2\_bitset.c

A

- 01101001 { 0, 3, 5, 6 } a set
- 76543210

B

- 01010101 { 0, 2, 4, 6 }
- 76543210

## ■ Operations

set operation between A and B

- & Intersection 01000001 { 0, 6 } Only elements present in **both** sets survive.
- | Union 01111101 { 0, 2, 3, 4, 5, 6 } Anything present in **either** set.
- ^ Symmetric difference 00111100 { 2, 3, 4, 5 } Elements in exactly one set, but not both.
- ~ Complement 10101010 { 1, 3, 5, 7 } not B

# Unix file permissions = 9 bits

```

r w x  r w x  r w x
u      g      o

```

Meaning:

- **u** = user (owner)
- **g** = group
- **o** = others

Each has:

- **r** = read
- **w** = write
- **x** = execute

Total = **9 bits**

**Example:**

```

r w x | r - x | r - -
1 1 1 | 1 0 1 | 1 0 0

```

Decimal: 754

## Why OS designers love bitsets

- extremely fast
- very compact
- easy to check (**&**)
- easy to modify (**|**, **~**)

# Bit-Level Operations in C

Bit-level operations treat numbers as *bit vectors*, not as math values.

## ■ Operations $\&$ , $|$ , $\sim$ , $\wedge$ Available in C

- Apply to any “integral” data type
  - long, int, short, char, unsigned
- View arguments as bit vectors
- Arguments applied bit-wise

not on float and double

Floating-point numbers have a different encoding – bitwise ops don’t make sense there.

## ■ Examples (Char data type)

- $\sim 0x41 \rightarrow 0xBE$ 
  - $\sim 01000001_2 \rightarrow 10111110_2$
- $\sim 0x00 \rightarrow 0xFF$ 
  - $\sim 00000000_2 \rightarrow 11111111_2$
- $0x69 \& 0x55 \rightarrow 0x41$ 
  - $01101001_2 \& 01010101_2 \rightarrow 01000001_2$
- $0x69 | 0x55 \rightarrow 0x7D$ 
  - $01101001_2 | 01010101_2 \rightarrow 01111101_2$

0101 NOT a number

0101 IS a pattern of bits

**Ox:** base-16, hexadecimal

**Ob:** base-2, binary

demo:

ch2\_bits\_ops.c

# Contrast: Logic Operations in C

## ■ Contrast to Logical Operators

■ `&&, ||, !`

Watch out for `&&` vs. `&` (and `||` vs. `|`)...  
one of the more common oopsies in  
C programming

- View 0 as “False”
- Anything nonzero as “True”
- Always return 0 or 1
- **Early termination**

Bitwise operators work on every bit.  
Logical operators work on “true or false”

In C logic:

- 0 = false
- anything nonzero = true
- result is always 0 or 1

## ■ Examples (char data type)

- `!0x41 → 0x00`
- `!0x00 → 0x01`
- `!!0x41 → 0x01`

! `!x` converts any value into strict 0 or 1  
“Double bang is how C normalizes truth.”

### Short-circuit rules:

false `&&` anything → false  
true `||` anything → true  
second operand **not evaluated**

- `0x69 && 0x55 → 0x01`
- `0x69 || 0x55 → 0x01`
- **`p && *p`** (avoids null pointer access)

# Why `p && *p` is written this way?

- Check that `p` is not `NULL` and the value it points to is nonzero.

- if (`p`): Does this pointer point somewhere valid?
- if (`*p`): the value stored at the address `p` points to

✗ Dangerous code  
// crash if `p == NULL`

- What happens in different cases?

- Case 1: `p == NULL`
- Case 2: `*p == 0`
- Case 3: `p != NULL`, and `*p != 0`

```
int x = 5; //x is stored at address 1000
int *p = &x;
```

```
// p → address of x
// *p → value of x
// if (p && *p): true only when both are true
```

- `*p && p` // ✗ WRONG ORDER

- `p = NULL` -> `*p` -> dereference `NULL` -> undefined behavior ->  
possible crash

- `&&` is not just `AND` — it's a guard.

demo:  
`ch2_null_pointer.c`

# Shift Operations

demo: ch2\_shift.c

## ■ Left Shift: $x \ll y$

- Shift bit-vector  $x$  left  $y$  positions
  - Throw away extra bits on left
  - Fill with 0's on right

Used for unsigned values

## ■ Right Shift: $x \gg y$

- Shift bit-vector  $x$  right  $y$  positions
  - Throw away extra bits on right
- Logical shift**
  - Fill with 0's on left
- Arithmetic shift**
  - Replicate most significant bit on left

|                |          |
|----------------|----------|
| Argument x     | 01100010 |
| $\ll 3$        | 00010000 |
| Log. $\gg 2$   | 00011000 |
| Arith. $\gg 2$ | 00011000 |

|                |          |
|----------------|----------|
| Argument x     | 10100010 |
| $\ll 3$        | 00010000 |
| Log. $\gg 2$   | 00101000 |
| Arith. $\gg 2$ | 11101000 |

## ■ Undefined Behavior

- Shift amount  $< 0$  or  $\geq$  word size

$x \ll 32$  // undefined for 32-bit int  
 $x \gg -1$  // undefined

# Integer Promotion

- In C, all integer types smaller than `int` are automatically promoted to `int` before arithmetic or bitwise operations.
  - `char` (1 byte)
  - `signed char` (1 byte)
  - `unsigned char` (1 byte)
  - `short` (2 bytes)
- example: `signed char s2 = 0xA2`; What happens when we evaluate `s2 >> 2`? (demo `ch2_shift.c`)
  - 8-bit `s2`: 10100010
  - 32-bit `int s2`: 11111111 11111111 11111111 10100010  
(promote to 32 bits)
  - `s2 >> 2`: 0xFFFFFFE8
  - Mask to 8 bits: 0xE8

# Today: Bits, Bytes, and Integers

- Representing information as bits
- Bit-level manipulations
- **Integers**
  - **Representation: unsigned and signed**
  - Conversion, casting
  - Expanding, truncating
  - Addition, negation, multiplication, shifting
  - Summary
- Representations in memory, pointers, strings
- Summary

# Encoding Integers

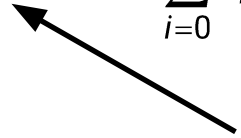
## Unsigned

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

## Two's Complement

$$B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

Sign  
Bit



```
short int x = 15213;
short int y = -15213;
```

### ■ C short 2 bytes long

Assuming a 16-bit short, bit 15 is the MSB/sign bit.

|          | Decimal | Hex   | Binary            |
|----------|---------|-------|-------------------|
| <b>x</b> | 15213   | 3B 6D | 00111011 01101101 |
| <b>y</b> | -15213  | C4 93 | 11000100 10010011 |

0x3B6D  
= 3×16<sup>3</sup> + B×16<sup>2</sup> + 6×16 + D

### ■ Sign Bit

For a 16-bit short: w-1=15

y = -1×2<sup>15</sup> + 1×2<sup>14</sup> + 1×2<sup>10</sup>  
+ 1×2<sup>7</sup> + 1×2<sup>4</sup> + 1×2 + 1×2<sup>0</sup>

- For 2's complement, **most significant bit (MSB)** indicates sign
  - 0 for nonnegative
  - 1 for negative

# Two-complement Encoding Example (Cont.)

**x** =           15213: 00111011 01101101  
**y** =           -15213: 11000100 10010011

**Unsigned:** all bit weights are positive.

**Two's complement signed:** same bits, but the leftmost bit has weight  $-2^{(w-1)}$ .

$w=16$

|            | Weight | 15213        |      | -15213        |        |           |
|------------|--------|--------------|------|---------------|--------|-----------|
| 1          | 1      | 1            | 1    | 1             | 1      | $2^0 = 1$ |
| 2          | 2      | 0            | 0    | 1             | 2      | $2^1 = 2$ |
| 3          | 4      | 1            | 4    | 0             | 0      | $2^2$     |
| 4          | 8      | 1            | 8    | 0             | 0      | $2^3$     |
| 5          | 16     | 0            | 0    | 1             | 16     | $2^4$     |
| 6          | 32     | 1            | 32   | 0             | 0      | $2^5$     |
| 7          | 64     | 1            | 64   | 0             | 0      | $2^6$     |
| 8          | 128    | 0            | 0    | 1             | 128    | $2^7$     |
| 9          | 256    | 1            | 256  | 0             | 0      | $2^8$     |
| 10         | 512    | 1            | 512  | 0             | 0      | $2^9$     |
| 11         | 1024   | 0            | 0    | 1             | 1024   | $2^{10}$  |
| 12         | 2048   | 1            | 2048 | 0             | 0      | $2^{11}$  |
| 13         | 4096   | 1            | 4096 | 0             | 0      | $2^{12}$  |
| 14         | 8192   | 1            | 8192 | 0             | 0      | $2^{13}$  |
| 15         | 16384  | 0            | 0    | 1             | 16384  | $2^{14}$  |
| 16         | -32768 | 0            | 0    | 1             | -32768 | $-2^{15}$ |
| <b>Sum</b> |        | <b>15213</b> |      | <b>-15213</b> |        |           |

# Numeric Ranges

## ■ Unsigned Values

$$\begin{aligned} \blacksquare \text{ } UMin &= 0 \\ &000\dots 0 \end{aligned}$$

$$\begin{aligned} \blacksquare \text{ } UMax &= 2^w - 1 \\ &111\dots 1 \end{aligned}$$

## Unsigned

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

## ■ Two's Complement Values

$$\begin{aligned} \blacksquare \text{ } TMin &= -2^{w-1} \\ &100\dots 0 \end{aligned}$$

$$\begin{aligned} \blacksquare \text{ } TMax &= 2^{w-1} - 1 \\ &011\dots 1 \end{aligned}$$

## ■ Other Values

$$\begin{aligned} \blacksquare \text{ } \text{Minus } 1 \\ &111\dots 1 \end{aligned}$$

### Values for $W = 16$

|      | Decimal       | Hex   | Binary            |
|------|---------------|-------|-------------------|
| UMax | <b>65535</b>  | FF FF | 11111111 11111111 |
| TMax | <b>32767</b>  | 7F FF | 01111111 11111111 |
| TMin | <b>-32768</b> | 80 00 | 10000000 00000000 |
| -1   | <b>-1</b>     | FF FF | 11111111 11111111 |
| 0    | <b>0</b>      | 00 00 | 00000000 00000000 |

$$TMax = 2^{15} - 1 = 32767$$

$$TMin = -2^{15} = -32768$$

**-1 = all 1s**

# Values for Different Word Sizes

|      | W    |         |                |                            |
|------|------|---------|----------------|----------------------------|
|      | 8    | 16      | 32             | 64                         |
| UMax | 255  | 65,535  | 4,294,967,295  | 18,446,744,073,709,551,615 |
| TMax | 127  | 32,767  | 2,147,483,647  | 9,223,372,036,854,775,807  |
| TMin | -128 | -32,768 | -2,147,483,648 | -9,223,372,036,854,775,808 |

## ■ Observations

- $|TMin| = TMax + 1$ 
  - Asymmetric range
- $UMax = 2 * TMax + 1$

## ■ C Programming

- `#include <limits.h>`
- Declares constants, e.g.,
  - `ULONG_MAX`
  - `LONG_MAX`
  - `LONG_MIN`
- Values platform specific

# Unsigned & Signed Numeric Values

| $X$  | $B2U(X)$ | $B2T(X)$ |
|------|----------|----------|
| 0000 | 0        | 0        |
| 0001 | 1        | 1        |
| 0010 | 2        | 2        |
| 0011 | 3        | 3        |
| 0100 | 4        | 4        |
| 0101 | 5        | 5        |
| 0110 | 6        | 6        |
| 0111 | 7        | 7        |
| 1000 | 8        | -8       |
| 1001 | 9        | -7       |
| 1010 | 10       | -6       |
| 1011 | 11       | -5       |
| 1100 | 12       | -4       |
| 1101 | 13       | -3       |
| 1110 | 14       | -2       |
| 1111 | 15       | -1       |

(important  
boundary case)  
 $-8 + 4 + 2 + 1 = -1$

## ■ Equivalence

- Same encodings for nonnegative values

## ■ Uniqueness

- Every bit pattern represents unique integer value
- Each representable integer has unique bit encoding

## ■ $\Rightarrow$ Can Invert Mappings

- $U2B(x) = B2U^{-1}(x)$ 
  - Bit pattern for unsigned integer
- $T2B(x) = B2T^{-1}(x)$ 
  - Bit pattern for two's comp integer

## Ex: Arithmetic right shift preserves what?

- A. Least significant bit
- B. Zeros
- C. Sign bit
- D. Carry bit

## Ex: 11111111 (8bit)

- A. 255 (unsigned)
- B. -1 (two's complement)
- C. Both A and B
- D. Undefined

# Ex: What is this in decimal? 0x7F

- A. 63
- B. 127
- C. 128
- D. -1

## Ex: What does %u mean in printf?

- A. Print in binary
- B. Print unsigned decimal
- C. Print hex
- D. Print Unicode

# Ex: predict output

```
#include <stdio.h>
#include <stdint.h>

int main(void)
{
    uint16_t u = 0xFFFF;
    int16_t s = (int16_t)0xFFFF;

    printf("%u\n", (unsigned int)u);
    printf("%d\n", (int)s);

    return 0;
}
```

A:  
65535  
65535

B:  
65535  
-1

C:  
-1  
-1

# different c types

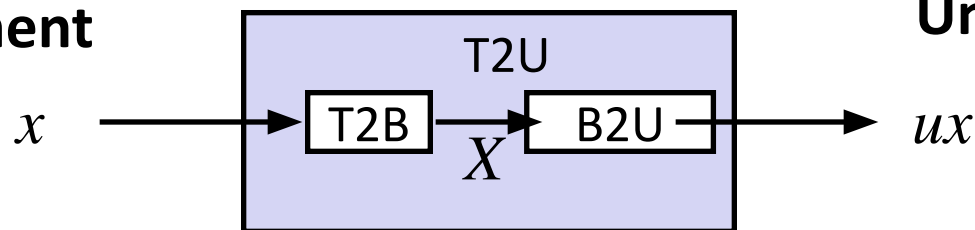
| Type                  | Meaning                 | Typical range                   |
|-----------------------|-------------------------|---------------------------------|
| <code>int8_t</code>   | signed 8-bit integer    | −128 to 127                     |
| <code>uint8_t</code>  | unsigned 8-bit integer  | 0 to 255                        |
| <code>int16_t</code>  | signed 16-bit integer   | −32,768 to 32,767               |
| <code>uint16_t</code> | unsigned 16-bit integer | 0 to 65,535                     |
| <code>int32_t</code>  | signed 32-bit integer   | −2,147,483,648 to 2,147,483,647 |
| <code>uint32_t</code> | unsigned 32-bit integer | 0 to 4,294,967,295              |

# Today: Bits, Bytes, and Integers

- Representing information as bits
- Bit-level manipulations
- **Integers**
  - Representation: unsigned and signed
  - **Conversion, casting**
  - Expanding, truncating
  - Addition, negation, multiplication, shifting
  - Summary
- Representations in memory, pointers, strings

# Mapping Between Signed & Unsigned

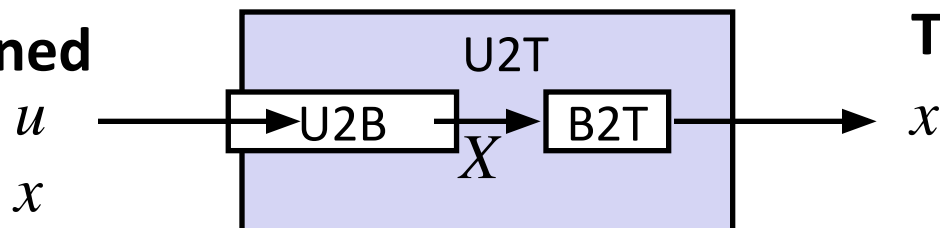
Two's Complement



Unsigned

Maintain Same Bit Pattern

Unsigned



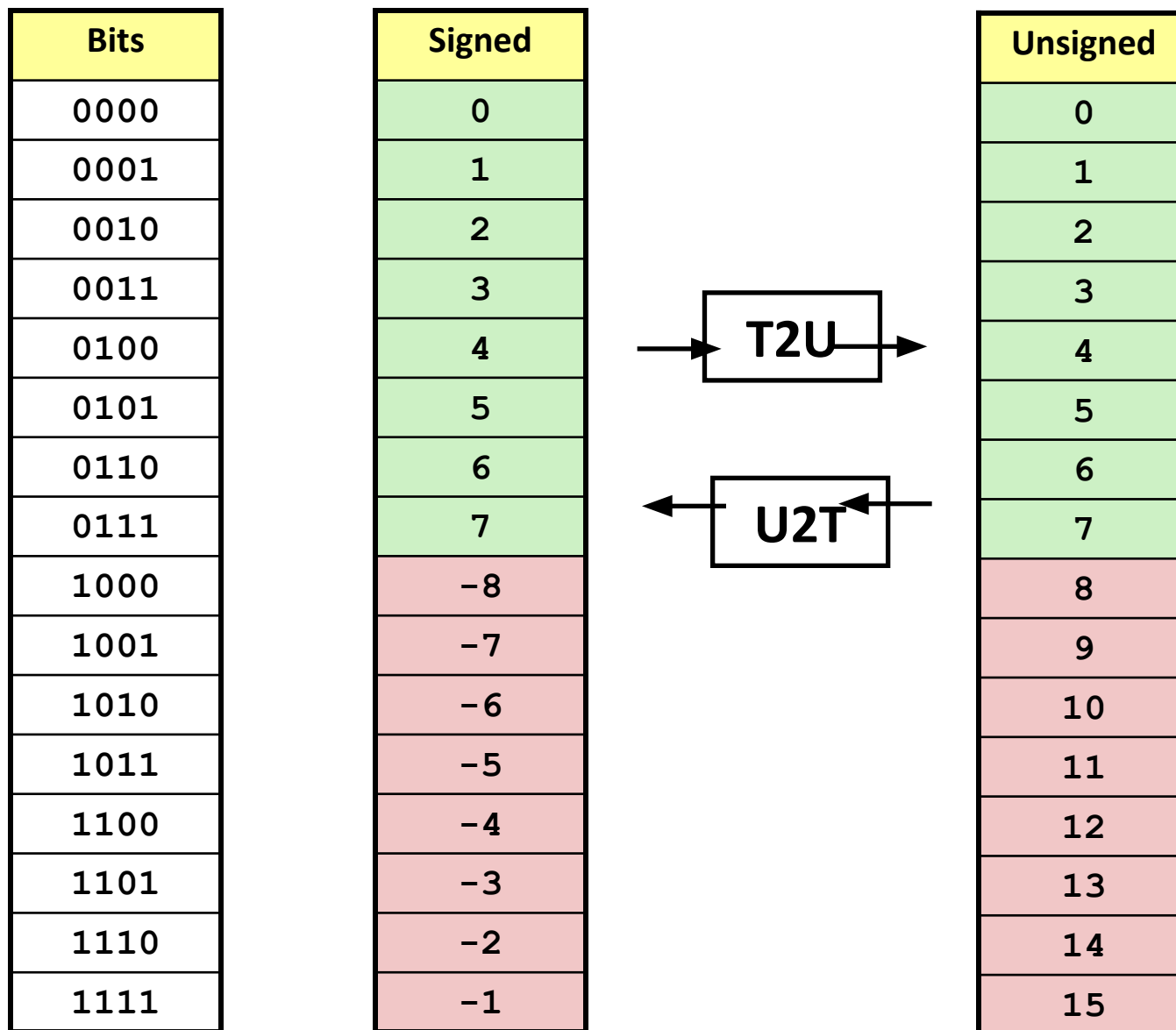
Two's Complement

Reinterpret

Maintain Same Bit Pattern

- Mappings between unsigned and two's complement numbers:  
**Keep bit representations and reinterpret**

# Mapping Signed $\leftrightarrow$ Unsigned

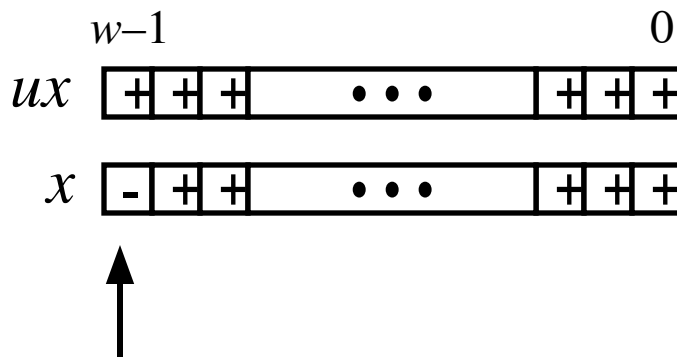
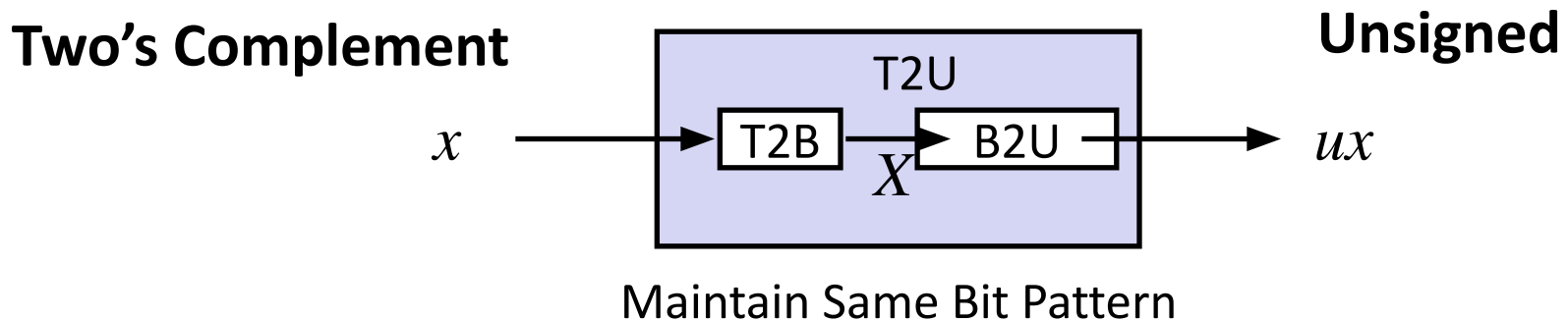


# Mapping Signed $\leftrightarrow$ Unsigned

| Bits | Signed |                                                                                                        | Unsigned |
|------|--------|--------------------------------------------------------------------------------------------------------|----------|
| 0000 | 0      |                     | 0        |
| 0001 | 1      |                                                                                                        | 1        |
| 0010 | 2      |                                                                                                        | 2        |
| 0011 | 3      |                                                                                                        | 3        |
| 0100 | 4      |                                                                                                        | 4        |
| 0101 | 5      |                                                                                                        | 5        |
| 0110 | 6      |                                                                                                        | 6        |
| 0111 | 7      |                                                                                                        | 7        |
| 1000 | -8     | <br><b>+/- 16</b> | 8        |
| 1001 | -7     |                                                                                                        | 9        |
| 1010 | -6     |                                                                                                        | 10       |
| 1011 | -5     |                                                                                                        | 11       |
| 1100 | -4     |                                                                                                        | 12       |
| 1101 | -3     |                                                                                                        | 13       |
| 1110 | -2     |                                                                                                        | 14       |
| 1111 | -1     |                                                                                                        | 15       |

**Signed = Unsigned - 16**  
**Unsigned = signed + 16**

# Relation between Signed & Unsigned



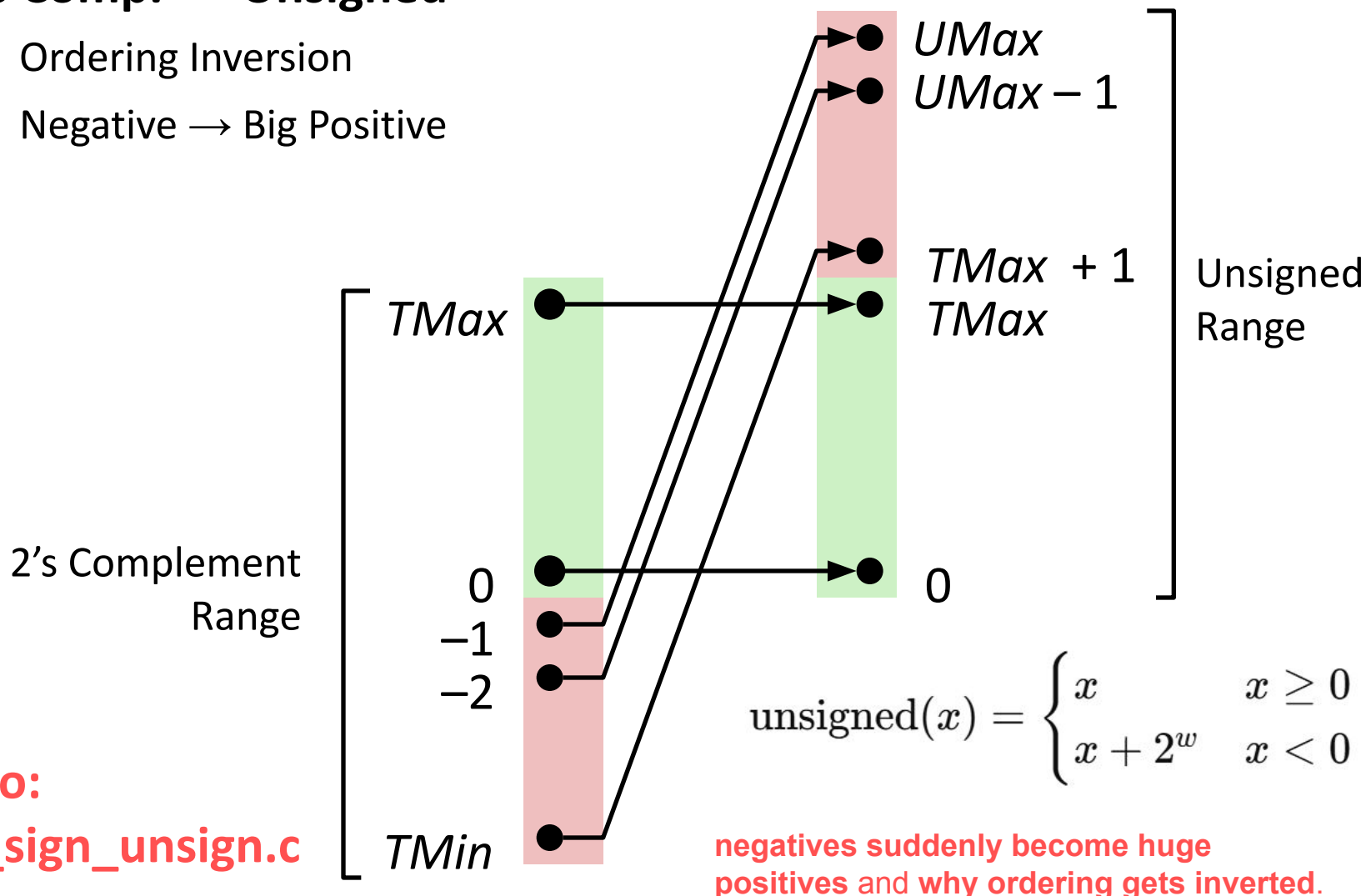
Large negative weight  
*becomes*  
 Large positive weight

- Bits don't change
- Interpretation changes
- Difference is always  $\pm 2^w$  for  $x < 0$

# Conversion Visualized

## ■ 2's Comp. → Unsigned

- Ordering Inversion
- Negative → Big Positive



demo:

ch2\_sign\_unsign.c

# Signed vs. Unsigned in C

## ■ Constants

- By default are considered to be **signed** integers
- Unsigned if have “U” as suffix

`0U, 4294967259U`

## ■ Casting

- **Explicit** casting between signed & unsigned same as U2T and T2U

```
int tx, ty;
unsigned ux, uy;
tx = (int) ux;
uy = (unsigned) ty;
```

- **Implicit** casting also occurs via assignments and procedure calls

```
tx = ux;
uy = ty;
```

# Casting Surprises

Casting first, and then Operation!

## ■ Expression Evaluation

- If there is a mix of unsigned and signed in single expression,  
*signed values implicitly cast to unsigned*
- Including comparison operations  $<$ ,  $>$ ,  $==$ ,  $<=$ ,  $>=$ , **arithmetic**  $(+ \ - \ * \ /)$
- Examples for  $W = 32$ : **TMIN = -2,147,483,648**, **TMAX = 2,147,483,647**

## ■ Constant<sub>1</sub> Constant<sub>2</sub> Relation Evaluation

0 0U == unsigned

-1 0 < signed

**-1 0U > unsigned** (unsigned)-1 = -1+2<sup>32</sup>

2147483647 -2147483647-1 > signed

**2147483647U -2147483647-1 < unsigned** (unsigned)(-2147483648) =  
-2147483648 + 2<sup>32</sup> = 2147483648U

-1 -2 > signed

**(unsigned)-1 -2 > unsigned** (unsigned)-1 = 4294967295

2147483647 2147483648U < unsigned 2147483647 = T<sub>max</sub> -> 2147483647U

**2147483647 (int) 2147483648U > signed** (int)2147483648U = -2147483648

# Values for Different Word Sizes

|      | W    |         |                |                            |
|------|------|---------|----------------|----------------------------|
|      | 8    | 16      | 32             | 64                         |
| UMax | 255  | 65,535  | 4,294,967,295  | 18,446,744,073,709,551,615 |
| TMax | 127  | 32,767  | 2,147,483,647  | 9,223,372,036,854,775,807  |
| TMin | -128 | -32,768 | -2,147,483,648 | -9,223,372,036,854,775,808 |

## ■ Observations

- $|TMin| = TMax + 1$ 
  - Asymmetric range
- $UMax = 2 * TMax + 1$

## ■ C Programming

- `#include <limits.h>`
- Declares constants, e.g.,
  - `ULONG_MAX`
  - `LONG_MAX`
  - `LONG_MIN`
- Values platform specific

# Does this operation cause signed integer overflow?

| Expression                    | Signed overflow?                                                                        | Explanation                                                                       |
|-------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>2147483647 + 1</code>   |  YES   | Computation in <code>int</code> exceeds <code>INT_MAX</code> → undefined behavior |
| <code>2147483648U + 1</code>  |  NO    | Unsigned arithmetic wraps by definition                                           |
| <code>(int)2147483648U</code> |  NO    | No arithmetic; just bit reinterpretation                                          |
| <code>-2147483647 - 1</code>  |  NO   | Result is <code>-2147483648</code> , still representable                          |
| <code>-2147483648 - 1</code>  |  YES | Signed underflow → undefined behavior                                             |

# Summary

## Casting Signed $\leftrightarrow$ Unsigned: Basic Rules

- Bit pattern is maintained
- But reinterpreted
- Can have unexpected effects: adding or subtracting  $2^w$
- Expression containing signed and unsigned int
  - `int` is cast to `unsigned`!! **Unsigned wins**

# Today: Bits, Bytes, and Integers

- Representing information as bits
- Bit-level manipulations
- **Integers**
  - Representation: unsigned and signed
  - Conversion, casting
  - **Expanding, truncating**
  - Addition, negation, multiplication, shifting
  - Summary
- Representations in memory, pointers, strings

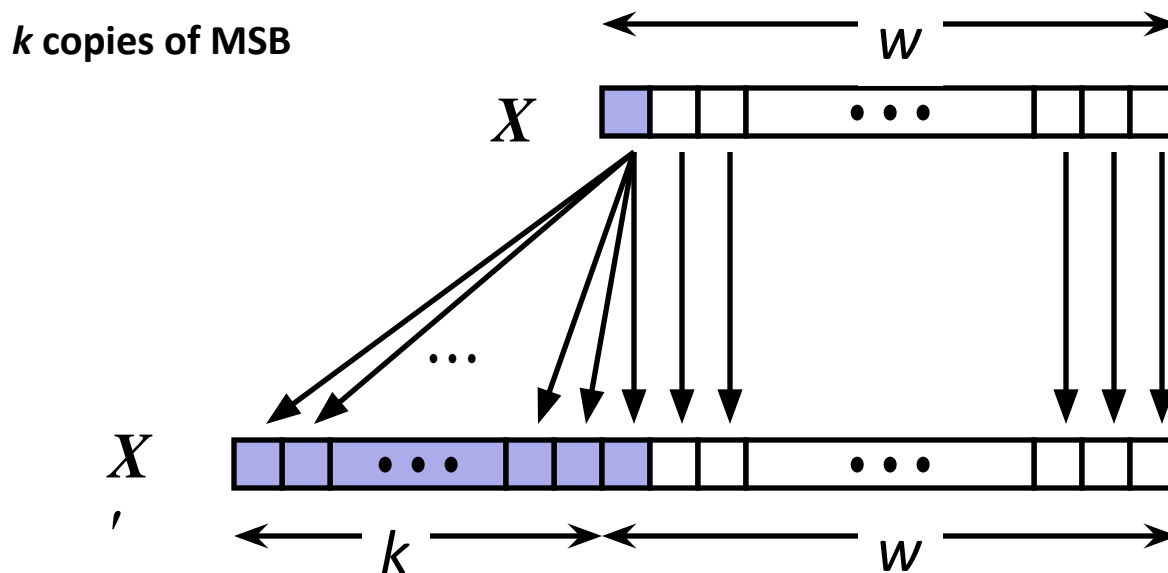
# Sign Extension

## ■ Task:

- Given  $w$ -bit signed integer  $x$
- Convert it to  $w+k$ -bit integer with same value

## ■ Rule:

- Make  $k$  copies of sign bit:
- $X' = \underbrace{x_{w-1}, \dots, x_{w-1}}_{k \text{ copies of MSB}}, x_{w-1}, x_{w-2}, \dots, x_0$



# Sign Extension Example

```
short int x = 15213;
int      ix = (int) x;
short int y = -15213;
int      iy = (int) y;
```

short is 2 byte, 16 bits  
int is 4 byte, 32 bits

|           | Decimal | Hex         | Binary                              |
|-----------|---------|-------------|-------------------------------------|
| <b>x</b>  | 15213   | 3B 6D       | 00111011 01101101                   |
| <b>ix</b> | 15213   | 00 00 3B 6D | 00000000 00000000 00111011 01101101 |
| <b>y</b>  | -15213  | C4 93       | 11000100 10010011                   |
| <b>iy</b> | -15213  | FF FF C4 93 | 11111111 11111111 11000100 10010011 |

- Converting from smaller to larger integer data type
- C automatically performs sign extension

# Summary:

## Expanding, Truncating: Basic Rules

### ■ Expanding (e.g., short int to int)

- Unsigned: zeros added
- Signed: sign extension
- Both yield expected result

expanding never loses information.

### ■ Truncating (e.g., unsigned to unsigned short)

- Unsigned/signed: bits are truncated
- Result reinterpreted
- Unsigned: **mod operation**
- Signed: **similar to mod**  $\text{result} = x \bmod 2^w$
- For small numbers yields expected behavior

truncating removes bits and reinterprets what remains.

demo: `ch2_expand.c`  
`ch2_truncate.c`

## Ex: predict output

```
unsigned short us = 0x00F0;  
unsigned int ui = us;
```

What is **ui** in decimal?

What is **ui** in hex? 0x000000F0

## Ex: Signed expansion

```
short s = -2;  
int i = s;
```

What is the hex value of **i**?

Why are the leftmost bits all **1**?

# Today: Bits, Bytes, and Integers

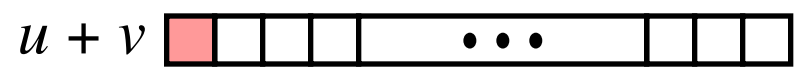
- Representing information as bits
- Bit-level manipulations
- **Integers**
  - Representation: unsigned and signed
  - Conversion, casting
  - Expanding, truncating
  - **Addition, negation, multiplication, shifting**
- Representations in memory, pointers, strings
- Summary

# Unsigned Addition

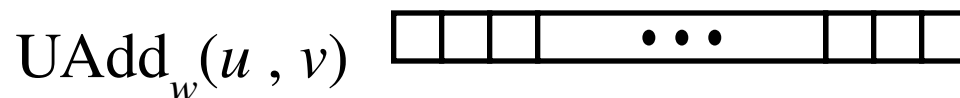
Operands:  $w$  bits



True Sum:  $w+1$  bits



Discard Carry:  $w$  bits



## ■ Standard Addition Function

- Ignores carry output

## ■ Implements Modular Arithmetic

$$s = \text{UAdd}_w(u, v) = (u + v) \bmod 2^w$$

keep only the **lowest  $w$  bits**  
values **wrap around** after  $2^w - 1$

not “normal math” in C — it is math on a fixed number of bits, and anything that doesn’t fit is thrown away.

8-bit:  $\bmod 2^8$  (256)  
16-bit:  $\bmod 2^{16}$  (65536)  
32-bit:  $\bmod 2^{32}$

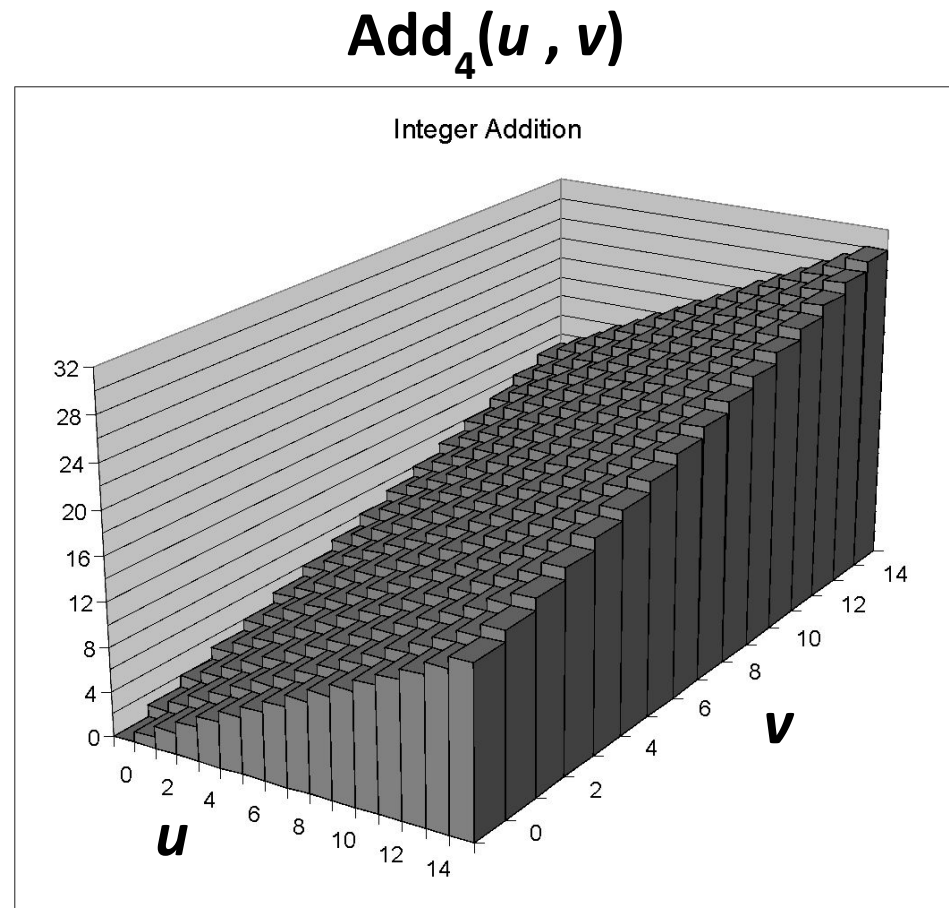
11111111 (255)  
+00000001 (1)  
**1 00000000** ← **needs 9 bits!**

255 + 1 = 0 (unsigned, 8-bit)

# Visualizing (Mathematical) Integer Addition

## ■ Integer Addition

- 4-bit integers  $u, v$
- Compute true sum  $\text{Add}_4(u, v)$
- Values increase linearly with  $u$  and  $v$
- Forms planar surface



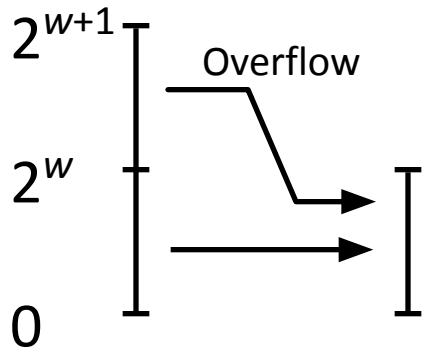
# Visualizing Unsigned Addition

demo:  
ch2\_unsigned\_add.c  
ch2\_true\_sum.c

## ■ Wraps Around

- If true sum  $\geq 2^w$
- At most once

True Sum



Modular Sum

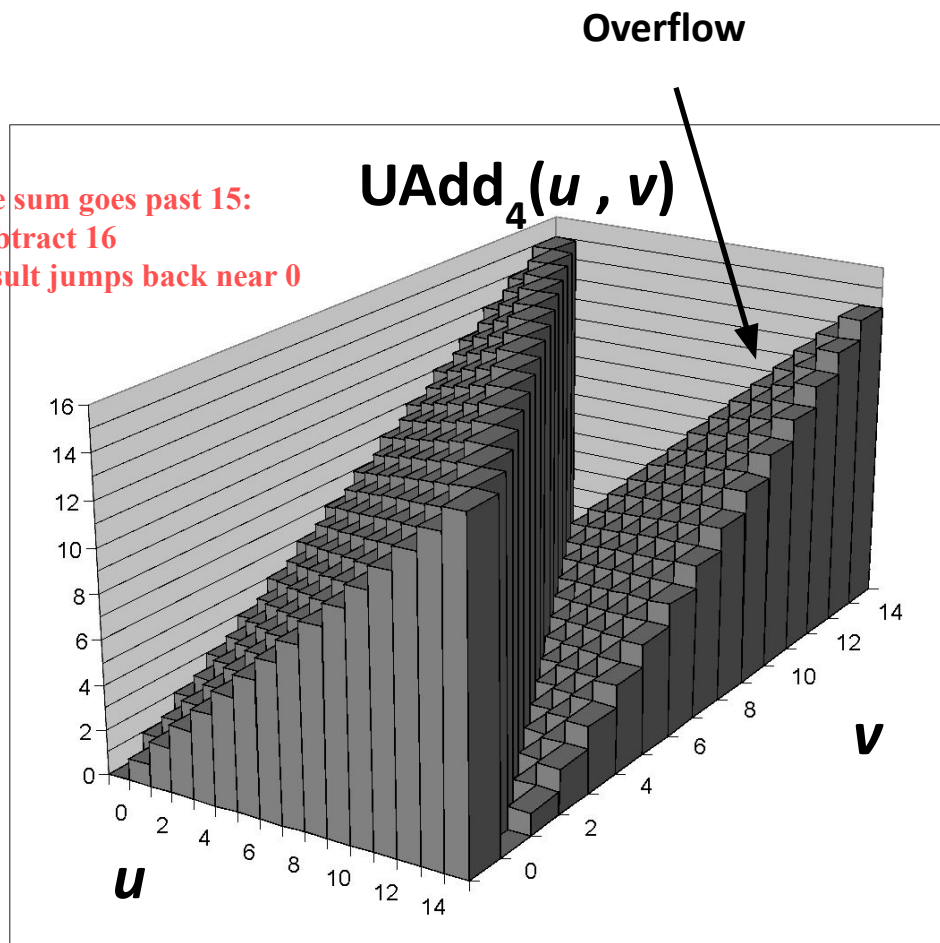
$$2^4 = 16$$

So:

If  $u + v < 16 \rightarrow$  normal

If  $u + v \geq 16 \rightarrow$  overflow  $\rightarrow$  wrap

- If the true sum goes past 15:
- subtract 16
  - result jumps back near 0

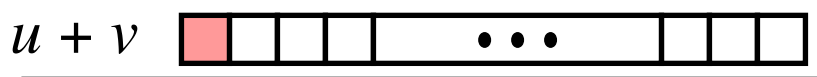


# Two's Complement Addition

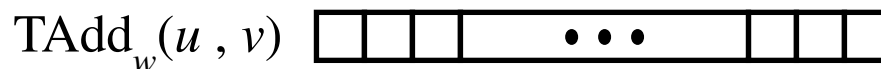
Operands:  $w$  bits



True Sum:  $w+1$  bits



Discard Carry:  $w$  bits



## ■ TAdd and UAdd have Identical Bit-Level Behavior

- Signed vs. unsigned addition in C:

```
int s, t, u, v;
```

```
s = (int) ((unsigned) u + (unsigned) v);
```

```
t = u + v
```

- Will give `s == t`

demo:

[ch2\\_unsigned\\_overflow.c](#)

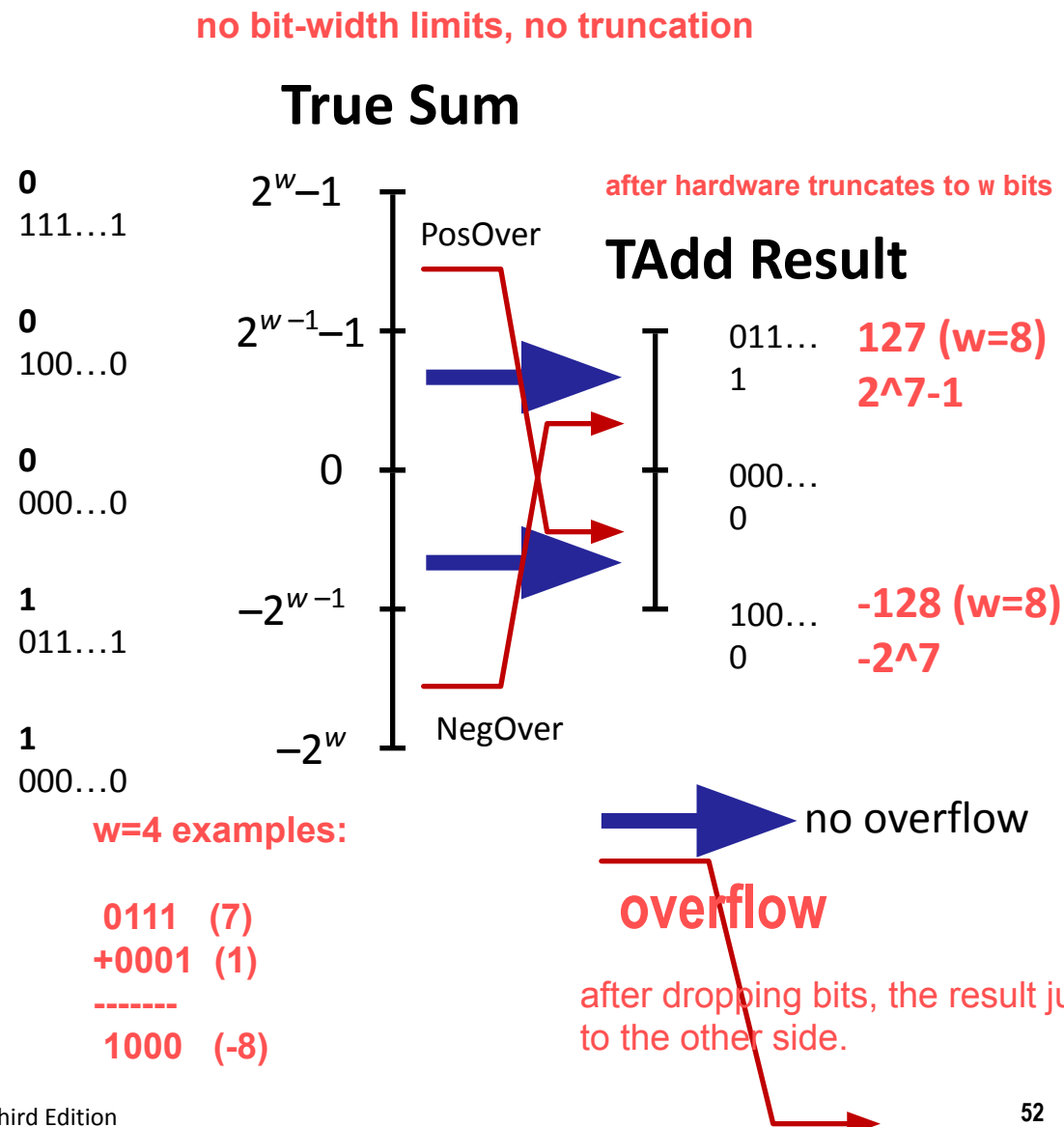
# TAdd Overflow

## ■ Functionality

- True sum requires  $w+1$  bits
- Drop off MSB
- Treat remaining bits as 2's comp. integer

unsigned:  $0 \dots 2^w - 1$

signed:  $-2^{w-1} \dots 2^{w-1} - 1$



# OVERFLOW (WRAPAROUND)

0111 (+7)  
+ 0111 (+7)

-----  
1110

1110 = -8+4+2 = -2

True sum = +14

Signed range max = +7

Overflow occurred

Stored result: -2

1000 (-8)  
+ 1000 (-8)

-----  
1 0000 ← true sum uses 5 bits  
0000 ← drop MSB

True sum = -16

Signed min = -8

Overflow occurred

Stored result: 0

When overflow happens:

- Big positive numbers wrap into negatives
- Big negative numbers wrap into positives

demo:

ch2\_signed\_overflow.c

# Visualizing 2's Complement Addition

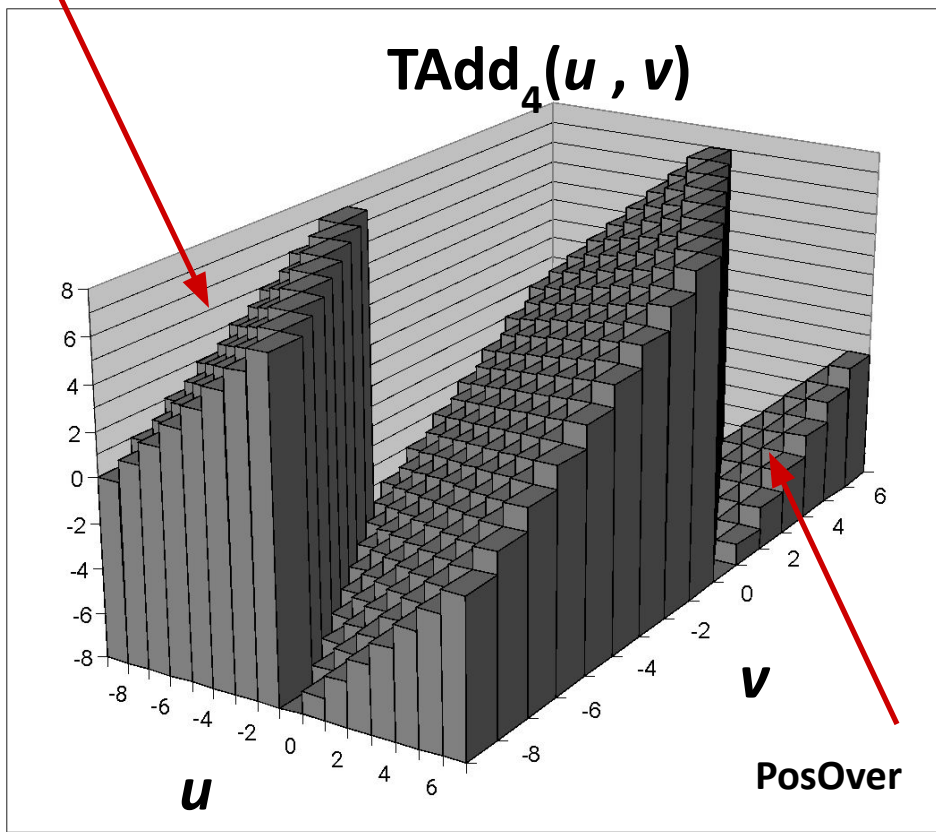
## ■ Values

- 4-bit two's comp.
- Range from -8 to +7

## ■ Wraps Around

- If  $\text{sum} \geq 2^{w-1} - 1$  (7)
  - Becomes negative
  - At most once
- If  $\text{sum} < -2^{w-1}$  (-8)
  - Becomes positive
  - At most once

NegOver



# Multiplication

exact multiplication cannot be done safely in  
fixed-width registers without overflow handling

## ■ Goal: Computing Product of $w$ -bit numbers $x, y$

- Either signed or unsigned

## ■ But, exact results can be bigger than $w$ bits

- Unsigned: up to  $2w$  bits

- Result range:  $0 \leq x * y \leq (2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$

$15 * 15 = 225$  (4bit \* 4bit = 8 bit)  
225 in binary = 11100001 (8 bit)

- Two's complement min (negative): Up to  $2w-1$  bits

- Result range:  $x * y \geq (-2^{w-1}) * (2^{w-1} - 1) = -2^{2w-2} + 2^{w-1}$

$-8 * -8 = 64$

$64 = 01000000$

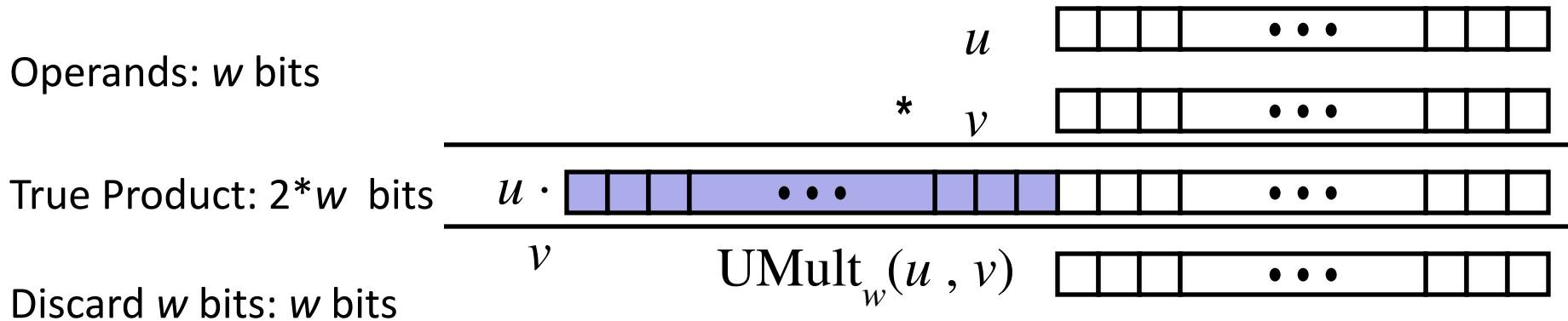
- Two's complement max (positive): Up to  $2w$  bits, but only for  $(TMin_w)^2$

- Result range:  $x * y \leq (-2^{w-1})^2 = 2^{2w-2}$

## ■ So, maintaining exact results...

- would need to keep expanding word size with each product computed
- is done in software, if needed
  - e.g., by “arbitrary precision” arithmetic packages

# Unsigned Multiplication in C



## ■ Standard Multiplication Function

- Ignores high order  $w$  bits

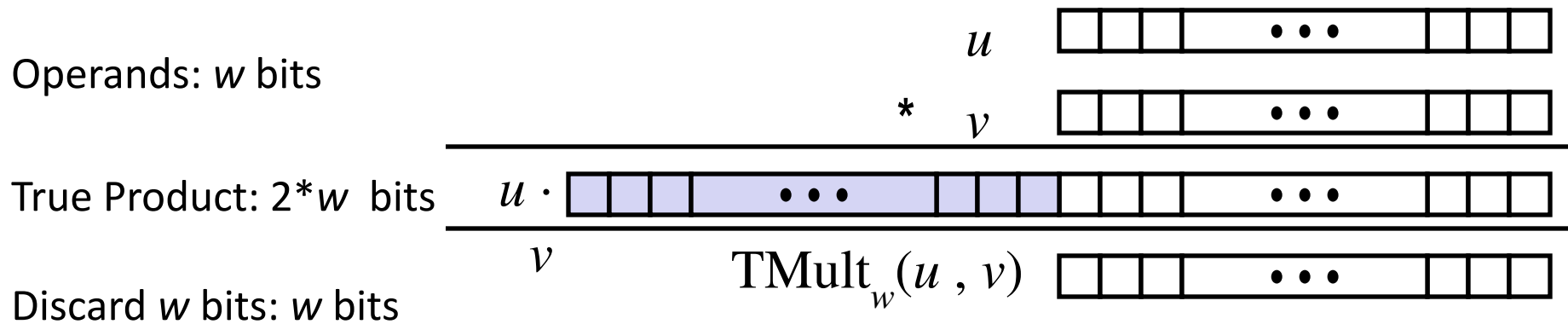
**C keeps only the low  $w$  bits of the true product**

## ■ Implements Modular Arithmetic

$$UMult_w(u, v) = u \cdot v \bmod 2^w$$

**demo:**  
**ch2\_multi.c**

# Signed Multiplication in C



## ■ Standard Multiplication Function

- Ignores high order  $w$  bits
- Some of which are different for signed vs. unsigned multiplication
- Lower bits are the same

**demo:**  
**ch2\_multi\_signed.c**

# Power-of-2 Multiply with Shift

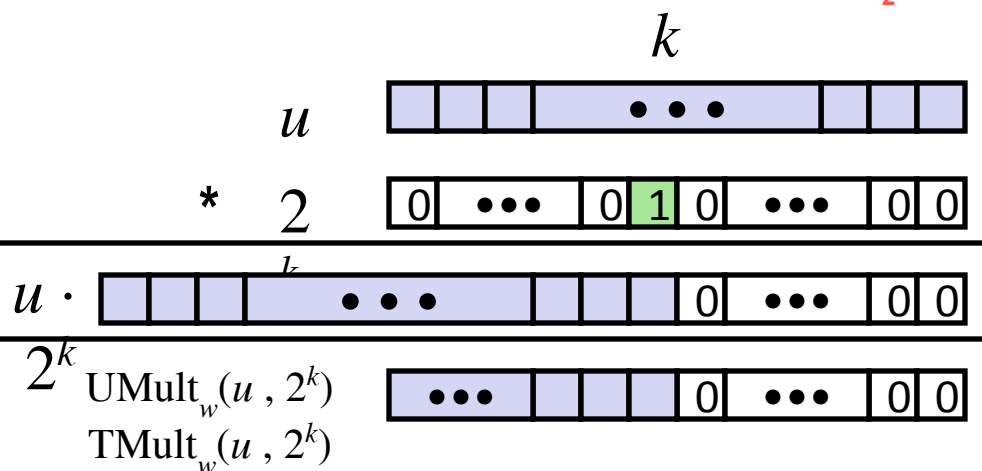
## ■ Operation

- $u \ll k$  gives  $u * 2^k$   $2^k$  (binary: 1 followed by  $k$  zeros),  $2^3 = 8 = 1000_2$
- Both signed and unsigned

Operands:  $w$  bits

True Product:  $w+k$  bits

Discard  $k$  bits:  $w$  bits



## ■ Examples

- $u \ll 3 == u * 8$
- $(u \ll 5) - (u \ll 3) == u * 24$  ( $u * 32 - u * 8$ )
- Most machines shift and add faster than multiply
  - Compiler generates this code automatically

# Unsigned Power-of-2 Divide with Shift

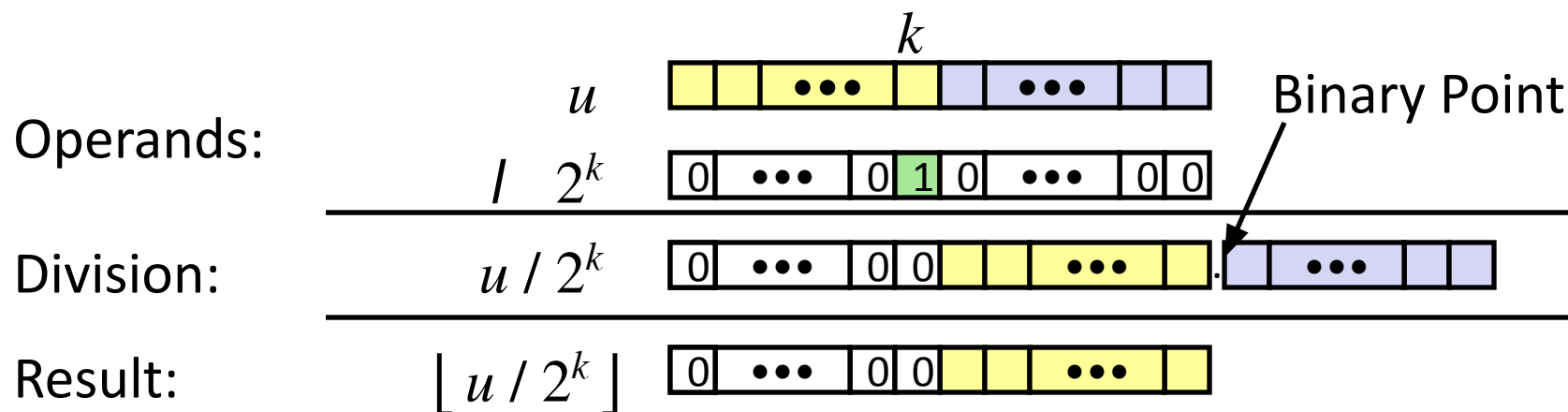
## ■ Quotient of Unsigned by Power of 2

demo: [ch2\\_devide.c](#)

■  $u \gg k$  gives  $\lfloor u / 2^k \rfloor$

$u \gg k \rightarrow$  shift right  $k$  bits, fill with 0s

■ Uses logical shift



automatically drops the fractional bits

|                     | Division          | Computed     | Hex   | Binary            |
|---------------------|-------------------|--------------|-------|-------------------|
| <b>x</b>            | <b>15213</b>      | <b>15213</b> | 3B 6D | 00111011 01101101 |
| <b>x &gt;&gt; 1</b> | <b>7606.5</b>     | <b>7606</b>  | 1D B6 | 00011101 10110110 |
| <b>x &gt;&gt; 4</b> | <b>950.8125</b>   | <b>950</b>   | 03 B6 | 00000011 10110110 |
| <b>x &gt;&gt; 8</b> | <b>59.4257813</b> | <b>59</b>    | 00 3B | 00000000 00111011 |

# Ex:

Which of the following conditions guarantees signed overflow?

- A. Adding two numbers with different signs
- B. Adding two positive numbers gives a negative result
- C. Adding two negative numbers gives a negative result
- D. Adding any numbers whose sum exceeds  $2^w$

# Ex:

Which statement is **true**?

- A. Signed and unsigned multiplication produce different low bits
- B. Unsigned multiplication is modulo  $2^w$
- C. Signed multiplication is modulo  $2^w$  in C
- D. Signed overflow is safe if hardware supports it

# Today: Bits, Bytes, and Integers

- Representing information as bits
- Bit-level manipulations
- **Integers**
  - Representation: unsigned and signed
  - Conversion, casting
  - Expanding, truncating
  - Addition, negation, multiplication, shifting
  - **Summary**
- Representations in memory, pointers, strings

# Arithmetic: Basic Rules

## ■ Addition:

- Unsigned/signed: Same low- $w$ -bit addition at the bit level
- Unsigned: addition mod  $2^w$ 
  - Overflow wraps around; defined in C
- Signed: normal addition if the result is representable
  - If signed overflow occurs, behavior is undefined in C
  - At the hardware/bit level, the low  $w$  bits may look like modulo- $2^w$  wraparound

## ■ Multiplication:

- Unsigned/signed: Same low- $w$ -bit result at the bit level
- Unsigned: multiplication mod  $2^w$ , overflow wraps around; defined in C
- Signed: normal multiplication if the result is representable
  - If signed overflow occurs, behavior is undefined in C

# Why Should I Use Unsigned?

## ■ *Don't* use without understanding implications

- Easy to make mistakes

```
unsigned i;  
for (i = cnt-2; i >= 0; i--)  
    a[i] += a[i+1];
```

- Can be very subtle

```
#define DELTA sizeof(int)  
int i;  
for (i = CNT; i-DELTA >= 0; i-= DELTA)  
    . . .
```

# Counting Down with Unsigned

## ■ Proper way to use unsigned as loop index

```
unsigned i;
for (i = cnt-2; i < cnt; i--)
    a[i] += a[i+1];
```

## ■ See Robert Seacord, *Secure Coding in C and C++*

- C Standard guarantees that unsigned addition will behave like modular arithmetic

- $0 - 1 \sqsubseteq UMax$

Unsigned arithmetic in C always wraps around modulo  $2^w$ , so subtracting 1 from 0 produces the maximum unsigned value.

## ■ Even better

```
size_t i;
for (i = cnt-2; i < cnt; i--)
    a[i] += a[i+1];
```

`size_t` is the unsigned type used by C to represent object sizes and array indices, and it safely matches the machine's word size.

`uint16_t` is about how many bits an integer has; `size_t` is about how much memory you can address.

- Data type `size_t` defined as unsigned value with length = word size
- Code will work even if `cnt = UMax`
- What if `cnt` is signed and `< 0`?

| Type                  | Purpose                                    |
|-----------------------|--------------------------------------------|
| <code>uint16_t</code> | Exactly 16-bit integer                     |
| <code>int8_t</code>   | Exactly 8-bit integer                      |
| <code>size_t</code>   | Large enough to index ANY object in memory |

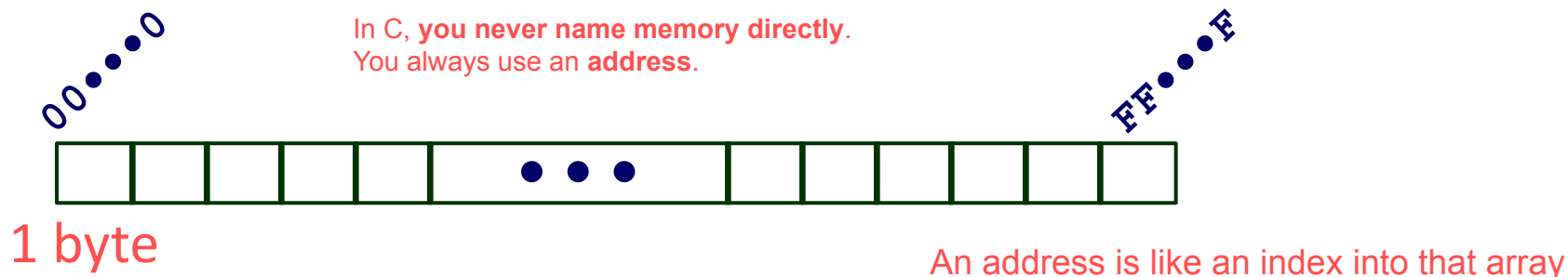
# Why Should I Use Unsigned? (cont.)

- **Do Use When Performing Modular Arithmetic**
  - Multiprecision arithmetic
- **Do Use When Using Bits to Represent Sets**
  - Logical right shift, no sign extension

# Today: Bits, Bytes, and Integers

- Representing information as bits
- Bit-level manipulations
- **Integers**
  - Representation: unsigned and signed
  - Conversion, casting
  - Expanding, truncating
  - Addition, negation, multiplication, shifting
  - Summary
- **Representations in memory, pointers, strings**

# Byte-Oriented Memory Organization



## ■ Programs refer to data by address

- Conceptually, envision it as **a very large array of bytes**
  - In reality, it's not, but can think of it that way
- An address is like an index into that array
  - and, a pointer variable stores an address

## ■ Note: system provides private address spaces to each “process”

- Think of a process as a program being executed
- So, a program can clobber its own data, but not that of others

Bugs like buffer overflows can corrupt **your own program**. But cannot (normally) corrupt another program or the OS

# Machine Words

A word is **not** a byte.  
It's usually **32 bits** or **64 bits**.

## ■ Any given computer has a “Word Size”

The machine word is the natural size for addresses and computation; it determines how much memory a computer can address, not the size of every data type.

- Nominal size of integer-valued data
  - and of addresses
- Until recently, most machines used 32 bits (4 bytes) as word size
  - Limits addresses to 4GB ( $2^{32}$  bytes) 2<sup>32</sup> possible addresses
- Increasingly, machines have 64-bit word size
  - Potentially, could have 18 EB (exabytes) of addressable memory
  - That's  $18.4 \times 10^{18}$  No computer actually has that much RAM — this is a **theoretical limit**.
- Machines still support multiple data formats
  - Fractions or multiples of word size
  - Always integral number of bytes

char, short, int have  
different sizes

# Word-Oriented Memory Organization

## ■ Addresses Specify Byte Locations

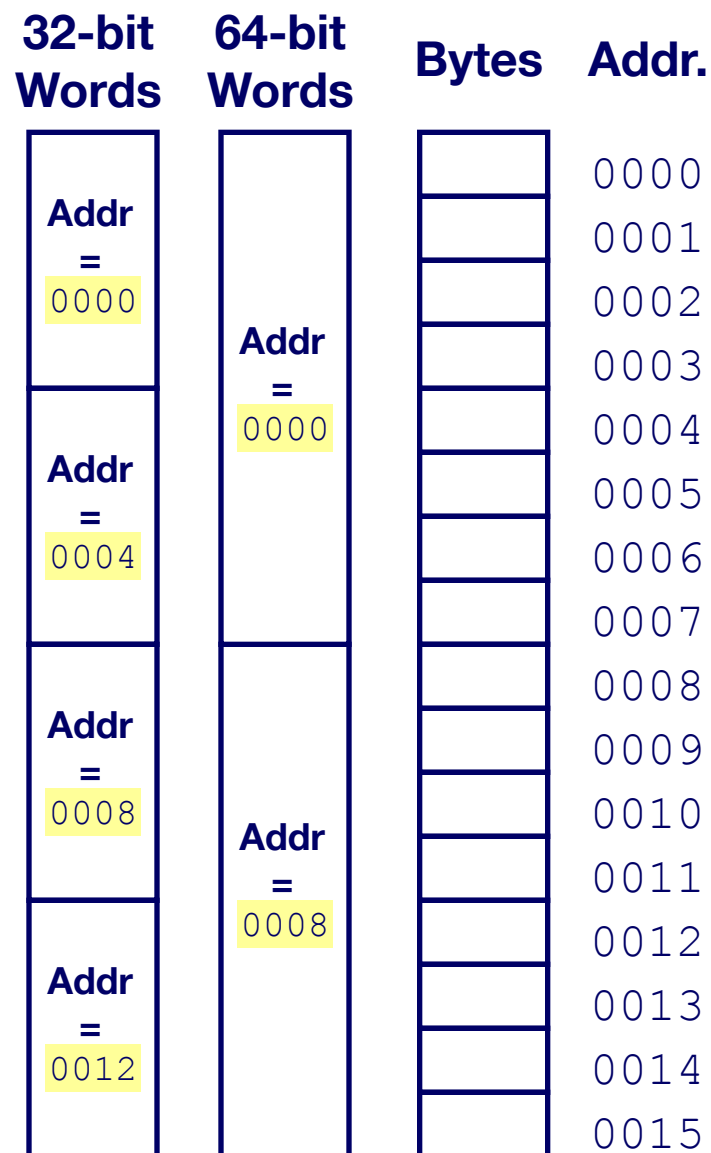
- Address of first byte in word
- Addresses of successive words differ by 4 (32-bit) or 8 (64-bit)

Memory is **byte-addressed**, *not* word-addressed.

A **word** is a group of bytes the CPU processes as a unit.

32-bit word = 4 bytes

64-bit word = 8 bytes



# Example Data Representations

| C Data Type              | Typical 32-bit | Typical 64-bit | x86-64 |
|--------------------------|----------------|----------------|--------|
| <code>char</code>        | 1              | 1              | 1      |
| <code>short</code>       | 2              | 2              | 2      |
| <code>int</code>         | 4              | 4              | 4      |
| <code>long</code>        | 4              | 8              | 8      |
| <code>float</code>       | 4              | 4              | 4      |
| <code>double</code>      | 8              | 8              | 8      |
| <code>long double</code> | –              | –              | 10/16  |
| <code>pointer</code>     | 4              | 8              | 8      |

# Byte Ordering

- So, how are the bytes within a multi-byte word ordered in memory?
- Conventions
  - Big Endian: Sun, PPC Mac, Internet
    - Least significant byte has highest address
  - Little Endian: x86, ARM processors running Android, iOS, and Windows
    - Least significant byte has lowest address

**Big endian:** “big part first”

**Little endian:** “little part first”

# Byte Ordering Example

## ■ Example

- Variable x has 4-byte value of 0x01234567
- Address given by &x is 0x100

### Little endian

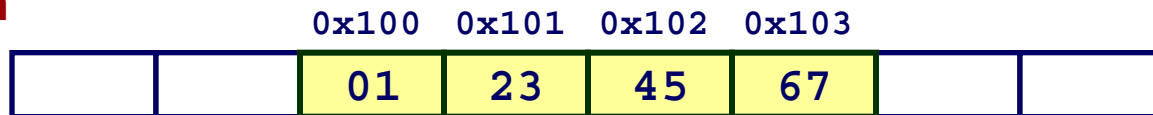
- 86 (Intel, AMD)
- ARM (Android, iOS, Windows)

→ most modern laptops and phones

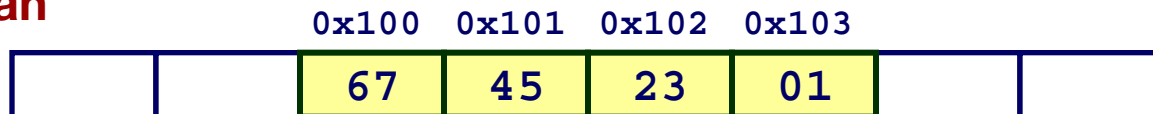
### Big endian

- Older systems
- Network protocols (Internet byte order)

### Big Endian



### Little Endian



Little endian stores numbers backwards

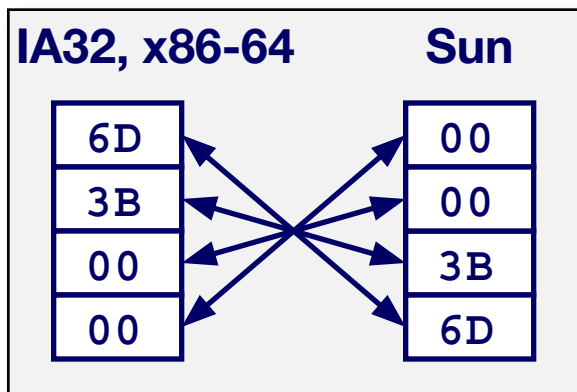
demo:  
ch2\_little\_endian.c

# Representing Integers

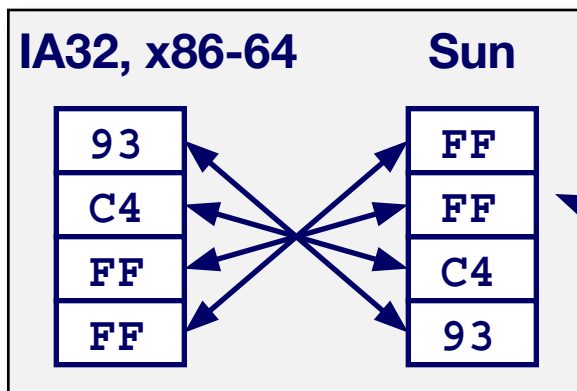
Endianness is about byte order, not bits

- Bits inside a byte do **not** change
- Only **byte sequence** changes

`int A = 15213;`



`int B = -15213;`

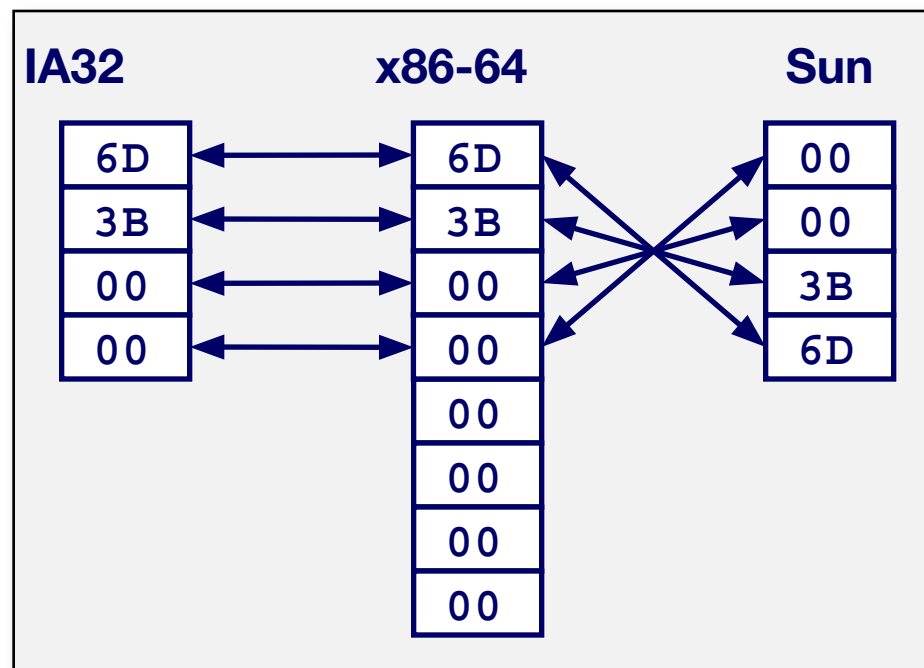


**Decimal:**      15213

**Binary:**      0011 1011 0110 1101

**Hex:**      3      B      6      D

`long int C = 15213;`



Same value, different memory layouts

Two's complement representation

# Examining Data Representations

## ■ Code to Print Byte Representation of Data

- Casting pointer to unsigned char \* allows treatment as a byte array

```
typedef unsigned char *pointer;

void show_bytes(pointer start, size_t len){
    size_t i;
    for (i = 0; i < len; i++)
        printf("%p\t0x%.2x\n", start+i, start[i]);
    printf("\n");
}
```

*start + i → address of byte i  
start[i] → value of that byte*

**demo:**  
**ch2\_15413.c**

**Printf directives:**  
%p: Print pointer  
%x: Print Hexadecimal

# show\_bytes Execution Example

```
int a = 15213;  
printf("int a = 15213;\n");  
show_bytes((pointer) &a, sizeof(int));
```

## Result (Linux x86-64):

```
int a = 15213;  
0x7fffb7f71dbc 6d  
0x7fffb7f71dbd 3b  
0x7fffb7f71dbe 00  
0x7fffb7f71dbf 00
```

# Representing Pointers

```
int B = -15213;  
int *P = &B;
```

| Sun | IA32 | x86-64 |
|-----|------|--------|
| EF  | AC   | 3C     |
| FF  | 28   | 1B     |
| FB  | F5   | FE     |
| 2C  | FF   | 82     |
|     |      | FD     |
|     |      | 7F     |
|     |      | 00     |
|     |      | 00     |

**Different compilers & machines assign different locations to objects**

**Even get different results each time run program**

# Representing Strings

- S is an array of **6 bytes**
- 5 bytes store characters
- 1 byte stores the **string terminator** `'\0'`

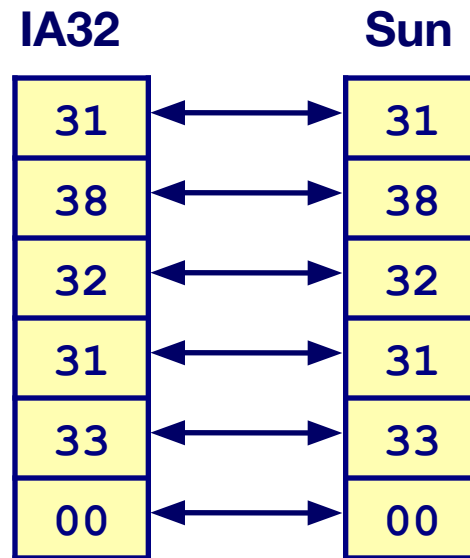
```
char S[6] = "18213";
```

## ■ Strings in C

- Represented by array of characters
- Each character encoded in ASCII format
  - Standard 7-bit encoding of character set
  - Character "0" has code 0x30
    - Digit *i* has code 0x30+*i*
- String should be null-terminated
  - Final character = 0

## ■ Compatibility

- Byte ordering not an issue



Is "18213" the same as 18213?

|           |     |     |     |     |     |      |
|-----------|-----|-----|-----|-----|-----|------|
| Address → | +0  | +1  | +2  | +3  | +4  | +5   |
| S[] holds | 31  | 38  | 32  | 31  | 33  | 00   |
|           | '1' | '8' | '2' | '1' | '3' | '\0' |

# Think about overflow!

Left-shifting a signed integer is undefined in C if the result overflows or changes the sign.

False •  $x < 0 \Rightarrow ((x*2) < 0)$  `int x = INT_MIN;`

True <sup>out</sup> • `ux`  $\geq 0$

False •  $x \& 7 == 7 \Rightarrow (x \ll 30) < 0$  only checks lowest 3 bits

False • `ux`  $> -1$  -1 -> 4294967295U

False •  $x > y \Rightarrow -x < -y$  y = INT\_MIN (-2^31)  
x = 0

False •  $x * x \geq 0$  -> -y = -INT\_MIN // overflow, unchanged

False •  $x > 0 \&\& y > 0 \Rightarrow x + y > 0$

True •  $x \geq 0 \Rightarrow -x \leq 0$

False •  $x \leq 0 \Rightarrow -x \geq 0$  x = INT\_MIN;

False •  $(x | -x) \gg 31 == -1$  Fails when x == 0

True • `ux`  $\gg 3 == ux/8$

False •  $x \gg 3 == x/8$  x = -7  
x >> 3 = -1

False •  $x \& (x-1) != 0$  x / 8 = 0

x = 8  
x & (x - 1) = 0

## Initialization

```
int x = foo();
```

```
int y = bar();
```

```
unsigned ux = x;
```

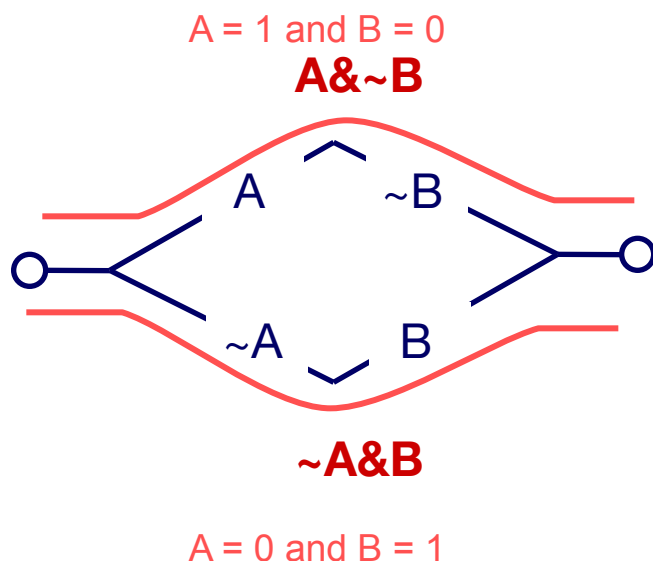
```
unsigned uy = y;
```

# Application of Boolean Algebra

## ■ Applied to Digital Systems by Claude Shannon

- 1937 MIT Master's Thesis
- Reason about networks of relay switches
  - Encode closed switch as 1, open switch as 0

XOR means “exactly one is true,” and Shannon showed you can build it with real switches.



Connection when

$$A \& \sim B \mid \sim A \& B$$

**^ exclusive OR**  
**=  $A \wedge B$**

| A | B | A & ~B | ~A & B | Output |
|---|---|--------|--------|--------|
| 0 | 0 | 0      | 0      | 0      |
| 0 | 1 | 0      | 1      | 1      |
| 1 | 0 | 1      | 0      | 1      |
| 1 | 1 | 0      | 0      | 0      |

# Binary Number Property

## Claim

$$1 + 1 + 2 + 4 + 8 + \dots + 2^{w-1} = 2^w$$

$$1 + \sum_{i=0}^{w-1} 2^i = 2^w \quad 1 + (2^w - 1) = 2^w$$

Example:  $w = 4$

$$1 + 1 + 2 + 4 + 8 = 16$$

- **$w = 0$ :**

- $1 = 2^0$

- **Assume true for  $w-1$ :**

- $1 + 1 + 2 + 4 + 8 + \dots + 2^{w-1} + 2^w = 2^w + 2^w = 2^{w+1}$



$= 2^w$

# Code Security Example

classic kernel security bug pattern

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes
   Never copy more than KSIZE (1024) */
/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

- Similar to code found in FreeBSD's implementation of `getpeername`
- There are legions of smart people trying to find vulnerabilities in programs

# Typical Usage

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

```
#define MSIZE 528

void getstuff() {
    char mybuf[MSIZE];
    copy_from_kernel(mybuf, MSIZE);
    printf("%s\n", mybuf);
}
```

# Malicious Usage

```
/* Declaration of library function memcpy */
void *memcpy(void *dest, void *src, size_t n);
```

```
/* Kernel memory region holding user-accessible data */
#define KSIZE 1024
char kbuf[KSIZE];

/* Copy at most maxlen bytes from kernel region to user buffer */
int copy_from_kernel(void *user_dest, int maxlen) {
    /* Byte count len is minimum of buffer size and maxlen */
    int len = KSIZE < maxlen ? KSIZE : maxlen;
    memcpy(user_dest, kbuf, len);
    return len;
}
```

- Tries to copy **4 GB** of kernel memory
- Reads far past kbuf
- Leaks kernel memory to user space

```
#define MSIZE 528

void getstuff() {
    char mybuf[MSIZE];
    copy_from_kernel(mybuf, -MSIZE);
    . . .
}
```

What if maxlen is negative?

```
len = 1024 < -528 ? 1024 : -528;
len = -528;
```

-528 → 0xfffffd0 (on 32-bit)  
That is **~4 GB of memory**

The bug:  
Is ONE missing check  
Only appears with malicious input

# Mathematical Properties

demo:  
ch2\_unsigned\_modular.c

## ■ Modular Addition Forms an *Abelian Group*

- **Closed** under addition

$$0 \leq \text{UAdd}_w(u, v) \leq 2^w - 1$$

If I add two 3-bit numbers, I always get another 3-bit number.

(w = 3 bits):  
6 + 5 = 11  
11 mod 8 = 3  
Still in range 0–7.

- **Commutative**

$$\text{UAdd}_w(u, v) = \text{UAdd}_w(v, u)$$

Order doesn't matter.

- **Associative**

$$\text{UAdd}_w(t, \text{UAdd}_w(u, v)) = \text{UAdd}_w(\text{UAdd}_w(t, u), v)$$

Parentheses don't matter

- **0** is additive identity

$$\text{UAdd}_w(u, 0) = u$$

Zero changes nothing.

- Every element has additive **inverse**

$$\begin{aligned} \text{Let } \text{UComp}_w(u) &= 2^w - u \\ \text{UAdd}_w(u, \text{UComp}_w(u)) &= 0 \end{aligned}$$

Every number has a “partner” that cancels it.

w = 3, 2<sup>w</sup> = 8  
u = 5, 8 - 5 = 3  
5 + 3 = 8 mod 8 = 0

# Mathematical Properties of TAdd

- **Isomorphic Group to unsigneds with UAdd**
  - $TAdd_w(u, v) = U2T(UAdd_w(T2U(u), T2U(v)))$ 
    - Since both have identical bit patterns
- Convert signed numbers to unsigned bit patterns
- Add them as unsigned (mod  $2^w$ )
- Interpret the result back as signed
- **Two's Complement Under TAdd Forms a Group**
  - Closed, Commutative, Associative, 0 is additive identity
  - Every element has additive inverse

$$TComp_w(u) = \begin{cases} -u & u \neq TMin_w \\ TMin_w & u = TMin_w \end{cases}$$

TMin is its own inverse.

demo:

ch2\_Isomorphic.c

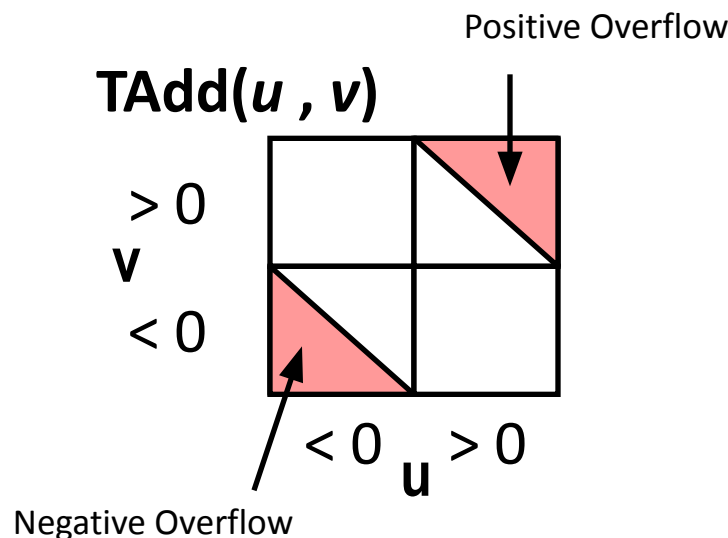
Signed addition is just modular arithmetic interpreted differently.

# Characterizing TAdd

demo:  
ch2\_Tadd.c

## ■ Functionality

- True sum requires  $w+1$  bits
- Drop off MSB
- Treat remaining bits as 2's comp. integer



Take the true mathematical sum  
If it doesn't fit in  $w$  bits  $\rightarrow$  drop the extra top bit  
Interpret the remaining bits as a signed number

On real machines (two's complement), this wrap happens

$$TAdd_w(u, v) = \begin{cases} u + v + 2^w & u + v < TMin_w \text{ (NegOver)} \\ u + v & TMin_w \leq u + v \leq TMax_w \\ u + v - 2^w & TMax_w < u + v \text{ (PosOver)} \end{cases}$$

# Negation: Complement & Increment

## ■ Claim: Following Holds for 2's Complement

$$\sim x + 1 == -x$$

$\sim x$ : Flip every single bit of  $x$ .

$-x$  is NOT the same as  $\sim x$

$0 \rightarrow 1$

$1 \rightarrow 0$

## ■ Complement

- Observation:  $\sim x + x == 1111\dots111 == -1$

$$\begin{array}{r}
 x \quad \boxed{1} \boxed{0} \boxed{0} \boxed{1} \boxed{1} \boxed{1} \boxed{0} \boxed{1} \\
 + \quad \sim x \quad \boxed{0} \boxed{1} \boxed{1} \boxed{0} \boxed{0} \boxed{0} \boxed{1} \boxed{0} \\
 \hline
 -1 \quad \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1} \boxed{1}
 \end{array}$$

Flip all bits

Add 1

## ■ Complete Proof?

$$\sim x + x = -1$$

$$x = 1000\ 0000 = -128$$

$$\text{flip: } 0111\ 1111$$

$$+1: 1000\ 0000: \text{It becomes itself again.}$$

$$-INT\_MIN == INT\_MIN$$

demo:

ch2\_5.c

ch2\_flip.c

# Complement & Increment Examples

**x = 15213**

|             | Decimal       | Hex   | Binary            |
|-------------|---------------|-------|-------------------|
| <b>x</b>    | <b>15213</b>  | 3B 6D | 00111011 01101101 |
| <b>~x</b>   | <b>-15214</b> | C4 92 | 11000100 10010010 |
| <b>~x+1</b> | <b>-15213</b> | C4 93 | 11000100 10010011 |
| <b>y</b>    | <b>-15213</b> | C4 93 | 11000100 10010011 |

**x = 0**

|             | Decimal   | Hex   | Binary            |
|-------------|-----------|-------|-------------------|
| <b>0</b>    | <b>0</b>  | 00 00 | 00000000 00000000 |
| <b>~0</b>   | <b>-1</b> | FF FF | 11111111 11111111 |
| <b>~0+1</b> | <b>0</b>  | 00 00 | 00000000 00000000 |

# Code Security Example #2

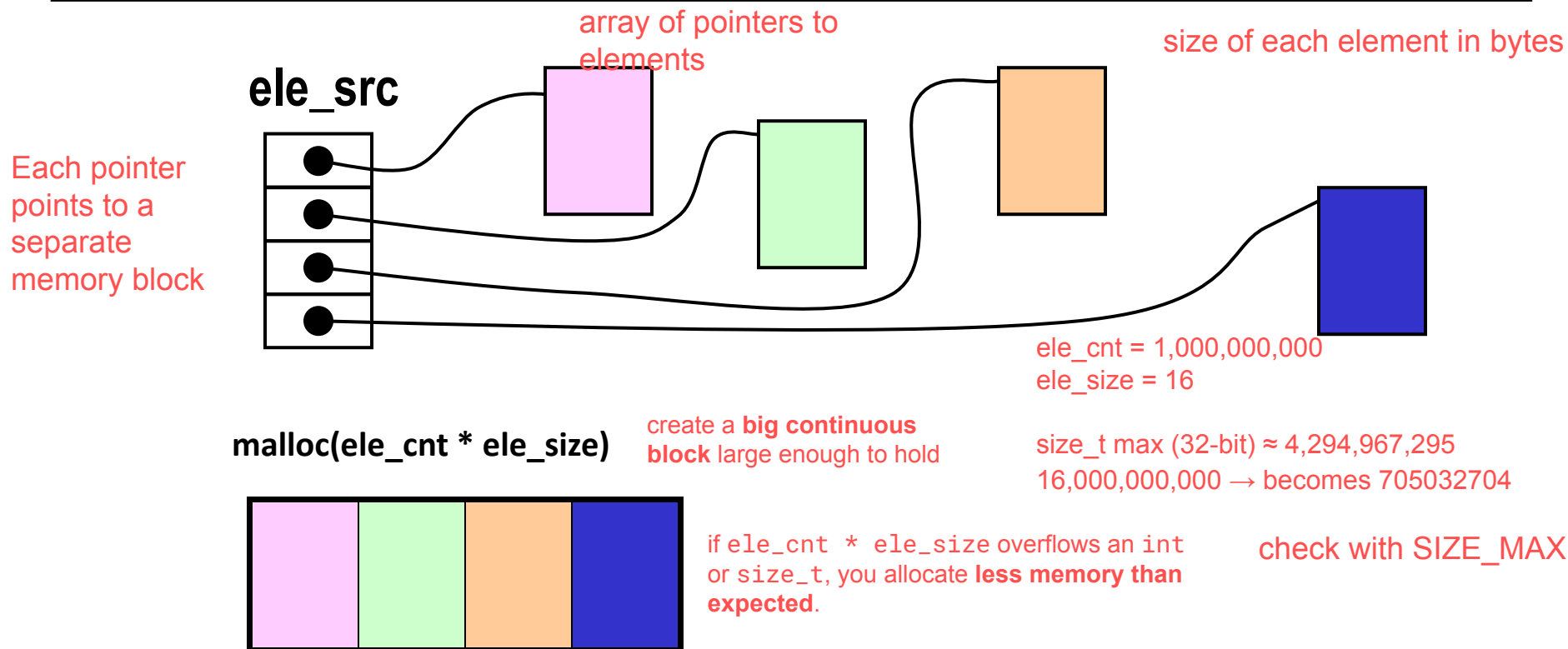
classic integer overflow → heap overflow security example.

## ■ SUN XDR library

- Widely used library for transferring data between machines

number of elements

```
void* copy_elements(void *ele_src[], int ele_cnt, size_t ele_size);
```



# XDR Code

1. Allocates memory for `ele_cnt` objects
2. Each object is `ele_size` bytes
3. Copies each object into a new contiguous block
4. Returns the pointer to that block

```
void* copy_elements(void *ele_src[], int ele_cnt, size_t ele_size) {  
    /*  
     * Allocate buffer for ele_cnt objects, each of ele_size bytes  
     * and copy from locations designated by ele_src  
     */  
    void *result = malloc(ele_cnt * ele_size);  
    if (result == NULL)  
        /* malloc failed */  
        return NULL;  
    void *next = result;  
    int i;  
    for (i = 0; i < ele_cnt; i++) {  
        /* Copy object i to destination */  
        memcpy(next, ele_src[i], ele_size);  
        /* Move pointer to next memory region */  
        next += ele_size;  
    }  
    return result;  
}
```

Memory corruption  
Possible crash  
Possible code execution

# XDR Vulnerability

`malloc(ele_cnt * ele_size)`

## ■ What if:

- `ele_cnt`      =  $2^{20} + 1$
- `ele_size`    = 4096              =  $2^{12}$
- Allocation = ??

A **heap overflow** (or heap buffer overflow) is a type of buffer overflow vulnerability that occurs when a program writes more data to a memory buffer located on the heap than the buffer is designed to hold.

`size_t max = 232 - 1`

$= 2^{20} * 2^{12} + 1 * 2^{12}$   
 $= 2^{32} + 2^{12}$   
 $= 2^{32} + 4096$   
 $\Rightarrow (2^{32} + 4096) \bmod 2^{32} = 4096$

## ■ How can I make this function secure?

```
if (ele_cnt < 0)
    return NULL;
```

```
if ((size_t)ele_cnt > SIZE_MAX / ele_size)
    return NULL;
```

```
size_t total = (size_t)ele_cnt * ele_size;
void *result = malloc(total);
```

# Compiled Multiplication Code

Multiplication by constant is just clever shifting and adding.

## C Function

```
long mul12(long x)
{
    return x*12;
}
```

$$\begin{aligned}
 &x * 12 \\
 &= x * (8 + 4) \\
 &= x*8 + x*4 \\
 &= (x \ll 3) + (x \ll 2)
 \end{aligned}$$

## Compiled Arithmetic Operations

```
leaq    (%rax,%rax,2), %rax
salq    $2, %rax
```

salq: arithmetic shift (preserve sign bit)

leap: logical shift (fill with 0)

## Explanation

```
t <- x+x*2
return t << 2;
```

- C compiler automatically generates shift/add code when multiplying by constant

- Shift is cheaper than multiplication
- Add is cheaper than multiplication
- Compiler optimizes constant multiplication

# Compiled Unsigned Division Code

## C Function

```
unsigned long udiv8
(unsigned long x)
{
    return x/8;
}
```

Division by 8 is just shifting right 3 bits.

## Compiled Arithmetic Operations

```
shrq    $3, %rax
```

## Explanation

```
# Logical shift
return x >> 3;
```

- Uses logical shift for unsigned
- For Java Users
  - Logical shift written as >>>

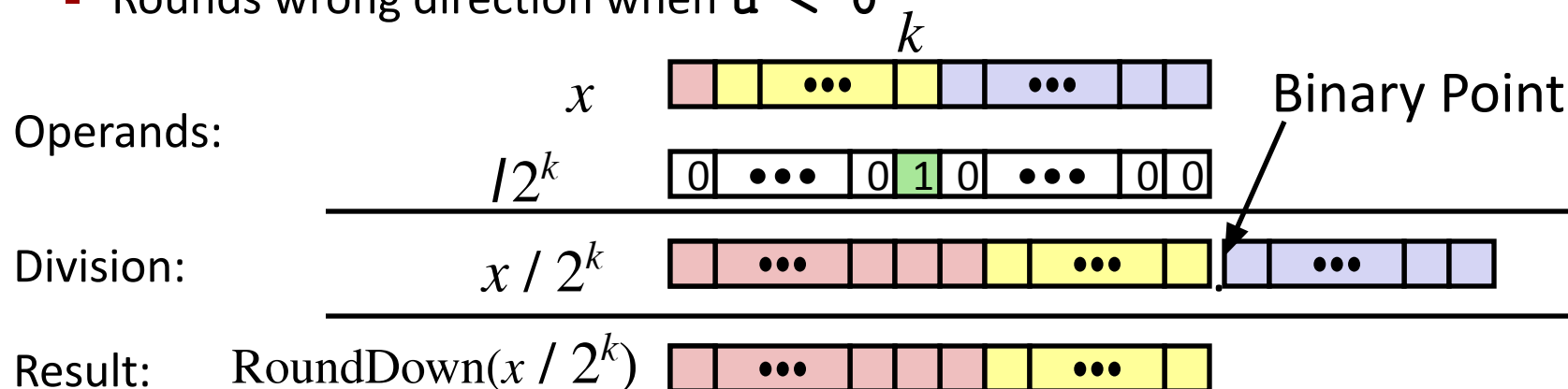
Signed → arithmetic shift

# Signed Power-of-2 Divide with Shift

## ■ Quotient of Signed by Power of 2

Copies sign bit (1 for negative numbers)

- $x \gg k$  gives  $\lfloor x / 2^k \rfloor$
- Uses arithmetic shift
- Rounds wrong direction when  $u < 0$



|           | Division    | Computed | Hex   | Binary            |
|-----------|-------------|----------|-------|-------------------|
| $y$       | -15213      | -15213   | C4 93 | 11000100 10010011 |
| $y \gg 1$ | -7606.5     | -7607    | E2 49 | 11100010 01001001 |
| $y \gg 4$ | -950.8125   | -951     | FC 49 | 11111100 01001001 |
| $y \gg 8$ | -59.4257813 | -60      | FF C4 | 11111111 11000100 |

# Correct Power-of-2 Divide

To divide negative numbers correctly with shift, you add a bias first. That bias moves negative numbers toward zero.

## ■ Quotient of Negative Number by Power of 2

- Want  $\lceil x / 2^k \rceil$  (Round Toward 0)
- Compute as  $\lfloor (x + 2^k - 1) / 2^k \rfloor$ 
  - In C:  $(x + (1 \ll k) - 1) \gg k$
  - Biases dividend toward 0

Multiply by  $2^k \rightarrow$  left shift

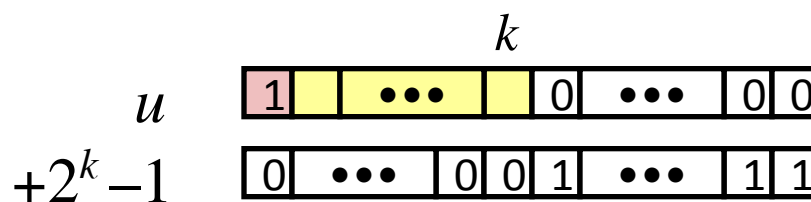
Unsigned divide  $\rightarrow$  logical shift

Signed divide (floor)  $\rightarrow$  arithmetic shift

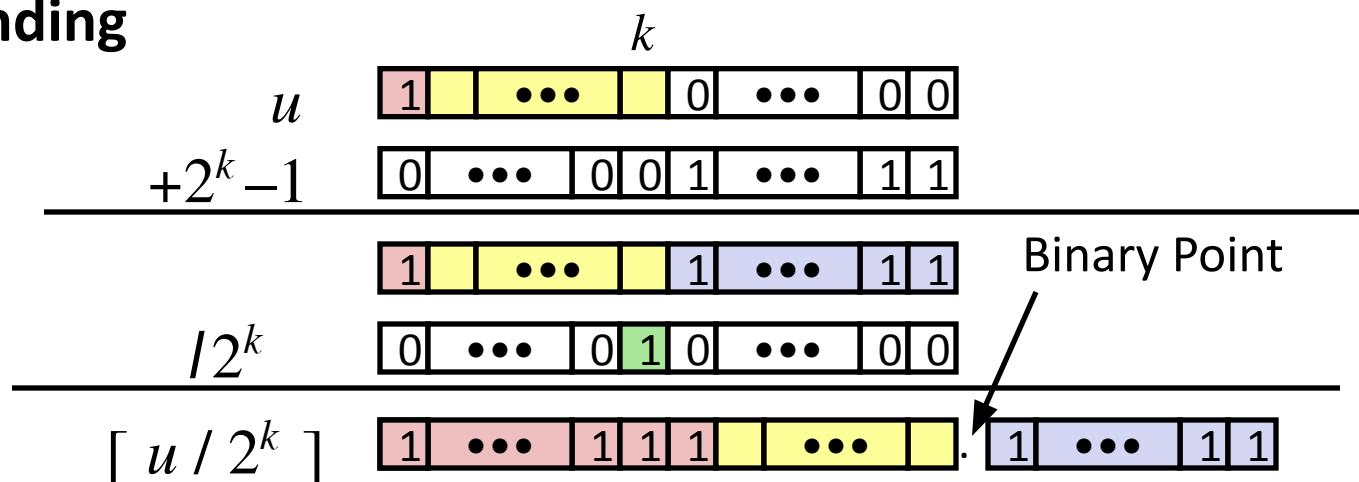
Signed divide (truncate)  $\rightarrow$  bias + shift

## Case 1: No rounding

Dividend:



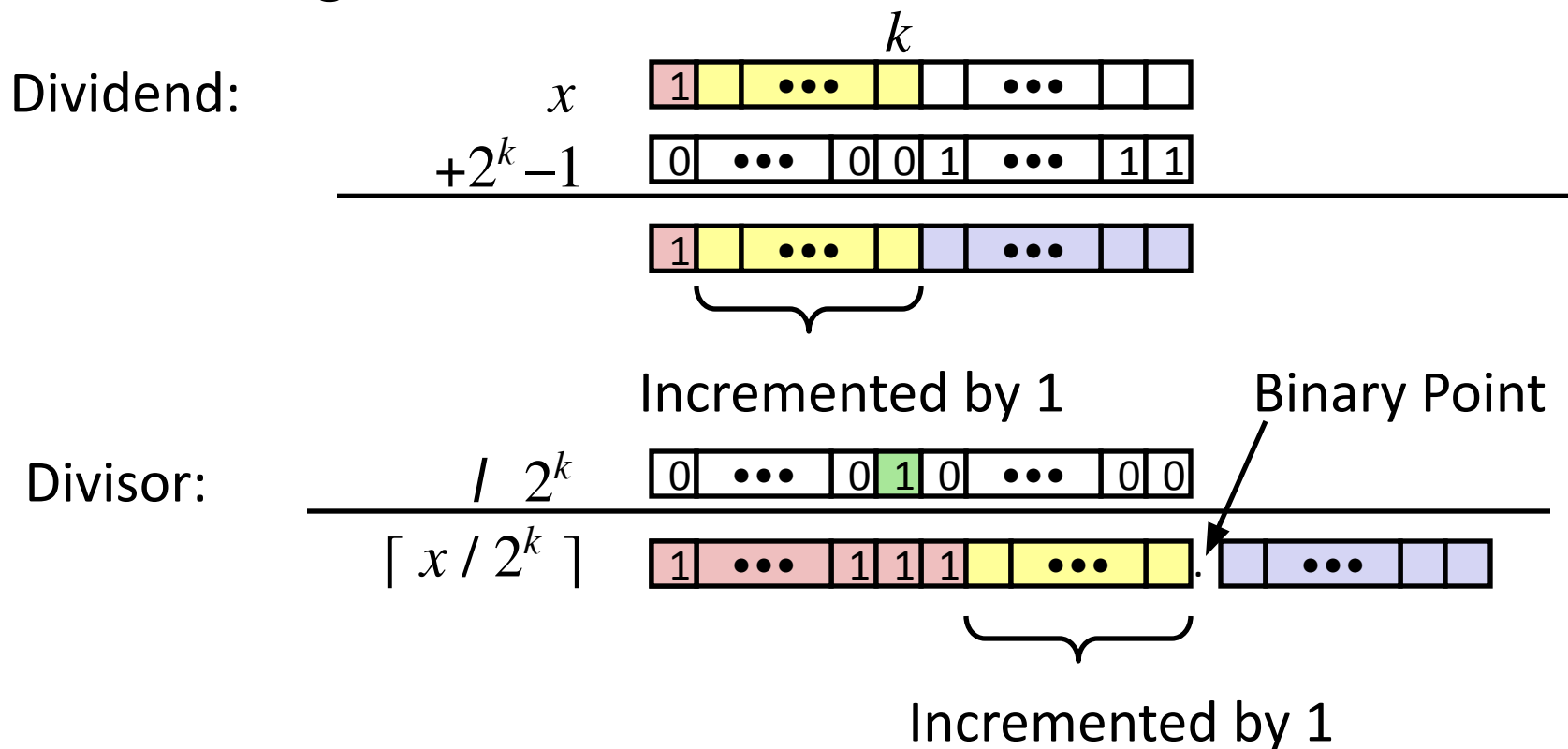
Divisor:



***Biasing has no effect: For positives***

# Correct Power-of-2 Divide (Cont.)

## Case 2: Rounding



***Biasing adds 1 to final result***

# Example:

To divide negative numbers correctly with shift, you add a bias first.

$x = -9$   
 $k = 2$

True C result:  $-9 / 4 = -2$

Arithmetic shift alone:  $-9 \gg 2 = -3$  (wrong)

So shift result is too small by 1. We must increment it.

now apply bias:  $2^2 - 1 = 3$   
 $(-9 + 3) \gg 2$   
 $= (-6) \gg 2$   
 $= -2$

# Compiled Signed Division Code

## C Function

```
long idiv8(long x)
{
    return x/8;
}
```

## Compiled Arithmetic Operations

```
    testq %rax, %rax
    js  L4
L3:
    sarq  $3, %rax
    ret
L4:
    addq  $7, %rax
    jmp  L3
```

## Explanation

```
    if x < 0
        x += 7;
    # Arithmetic shift
    return x >> 3;
```

- Uses arithmetic shift for int
- For Java Users

- Arith. shift written as >>

# Arithmetic: Basic Rules

- **Unsigned ints, 2's complement ints are isomorphic rings:**  
**isomorphism = casting**

Unsigned and signed integers use  
the same bit patterns.  
The only difference is how we  
interpret them.

11111111  
Unsigned: 255  
Signed: -1

- **Left shift <<**

- Unsigned/signed: multiplication by  $2^k$
- Always logical shift

- **Right shift >>**

- Unsigned: logical shift, div (division + round to zero) by  $2^k$
- Signed: arithmetic shift
  - **Positive** numbers: div (division + round to zero) by  $2^k$
  - **Negative** numbers: div (division + round away from zero) by  $2^k$   
Use biasing to fix

# Properties of Unsigned Arithmetic

## ■ Unsigned Multiplication with Addition Forms

### Commutative Ring

Unsigned arithmetic wraps around, but still obeys all the usual algebra rules.

- Addition is commutative group

- Closed under multiplication

$$0 \leq \text{UMult}_w(u, v) \leq 2^w - 1$$

- Multiplication Commutative

$$\text{UMult}_w(u, v) = \text{UMult}_w(v, u)$$

- Multiplication is Associative

$$\text{UMult}_w(t, \text{UMult}_w(u, v)) = \text{UMult}_w(\text{UMult}_w(t, u), v)$$

- 1 is multiplicative identity

$$\text{UMult}_w(u, 1) = u$$

- Multiplication distributes over addition

$$\text{UMult}_w(t, \text{UAdd}_w(u, v)) = \text{UAdd}_w(\text{UMult}_w(t, u), \text{UMult}_w(t, v))$$

# Properties of Two's Comp. Arithmetic

## ■ Isomorphic Algebras

- Unsigned multiplication and addition
  - Truncating to  $w$  bits
- Two's complement multiplication and addition
  - Truncating to  $w$  bits

They differ only in interpretation of the bit patterns.

## ■ Both Form Rings

- Isomorphic to ring of integers mod  $2^w$

Computers don't do integer arithmetic. They do modular arithmetic. And we pretend it's normal integers — until overflow reminds us it isn't.

## ■ Comparison to (Mathematical) Integer Arithmetic

- Both are rings
- Integers obey ordering properties, e.g.,

$$u > 0 \quad \Rightarrow \quad u + v > v$$

$$u > 0, v > 0 \quad \Rightarrow \quad u \cdot v > 0$$

- These properties are not obeyed by two's comp. arithmetic

$$TMax + 1 == TMin$$

$$15213 * 30426 == -10030 \text{ (16-bit words)}$$